目录

TermSync：终端协同代码编辑器 **4**

1 项目概述 4

2 项目背景 6

3 功能描述 7

3.1 多人连接与文档同步 7

3.2 字符级编辑操作 7

3.3 ACK 驱动的客户端发送队列 8

3.4 服务端主导 OT 与全局版本 8

3.5 保存、自动保存与编辑日志 9

3.6 历史裁剪与重同步 10

4 设计思路 10

4.1 总体架构 10

4.2 网络协议设计 10

4.3 文档锁与全局顺序 11

4.4 OT 位置变换规则 11

4.5 客户端状态机 12

4.6 自动保存进程 12

5 测试与验证 12

5.1 单元测试 12

5.2 本地验证 13

5.3 部署验证 16

6 问题与解决方案 19

6.1 并发操作顺序不确定 19

6.2 旧版本操作位置失效 19

6.3 连续输入容易丢失或乱序 19

6.4 慢客户端可能阻塞服务端 20

6.5 历史无限增长 20
6.6 文件保存可能覆盖旧内容 20
6.7 中文字符显示宽度 20
7 项目不足与改进方向 20
8 代码展示 21
8.1 JSON Lines 协议封装 21
8.2 编辑操作模型 23
8.3 服务端 OP 日志输出 24
8.4 OT 位置变换规则 24
8.5 服务端权威文档处理流程 25
8.6 服务端锁外发送与广播 28
8.7 客户端发送队列与 ACK 处理 30
8.8 终端按键处理 32
8.9 自动保存与备份 34
8.10 文档一致性测试 36
8.11 客户端队列测试 38

Linux 平台下的基本命令 41

1 文件和目录操作 41
2 文件内容查看与处理命令 42
3 系统信息查看命令 44
4 权限相关命令 47
5 管道符组合命令 47
6 实验小结 49

Linux 下的 Shell 编程案例 50

1 程序功能说明 50
2 Shell 脚本执行方法 50
3 程序实现思路 51

4 案例展示 51
5 Shell 程序调试方法 55
6 实验小结 58
7 源代码展示 58

TermSync：终端协同代码编辑器

1 项目概述

本项目名为 TermSync，是一个使用 Python 标准库实现的终端多人协作代码编辑器。项目的目标不是复刻完整的图形化 IDE，而是在 Linux 环境下实现一个可实际运行、可多人连接、可同步编辑、可保存和可测试验证的轻量级协作编辑系统。用户在终端中运行客户端，通过 TCP 连接到服务端；服务端维护唯一权威文档状态，并负责对所有编辑操作进行排序、变换、应用和广播。

我刻意减少第三方依赖，主要使用 Python 标准库完成系统实现。网络通信基于 `socket`，并发处理使用 `threading` 和 `multiprocessing`，终端界面使用 `curses`，消息格式和持久化则基于 `json`、`pathlib` 等标准库模块。项目没有依赖现成的 Web 框架、协同编辑库或数据库系统，因此网络协议、客户端状态机、服务端 OT 变换、广播同步和保存机制都由项目代码自行实现，更能体现 Linux 程序设计中对进程、线程、文件和网络等基础能力的直接使用。

本项目覆盖了 Linux 程序设计中的多个核心主题：网络编程、线程创建与同步、进程间通信、文件系统操作、终端界面编程和自动化测试。服务端使用 socket 监听客户端连接，为每个客户端创建独立线程；文档对象使用锁保护共享状态；自动保存和日志写入被拆分到独立进程中，通过 `multiprocessing.Queue` 与主服务端通信；客户端使用 `curses` 构建终端交互界面。

项目采用“服务端主导 OT（Operational Transformation）”的同步方案。客户端只负责将键盘输入转换为字符级编辑操作，并按 ACK 节奏发送到服务端。服务端作为唯一权威状态源，在临界区内为每个操作分配递增的 `server_version`，并根据历史操作对旧版本操作进行位置变换。这样既降低了客户端实现复杂度，也能清晰展示并发编辑时的一致性处理过程。

在实际部署中，服务端运行在我的个人 Linux 服务器上。这台服务器是拥有独立公网 IP 的阿里云 ECS 服务器，并已经绑定域名 `glance02.xyz`，因此客户端后续不需要直接记忆服务器公网 IP，而是通过域名连接服务端。例如服务端在 Linux 服务器上监听 `0.0.0.0:8765`，客户端可以使用 `glance02.xyz:8765` 加入同一个协作文档。由于我的两台电脑都是 Windows 系统，客户端实际运行在 Windows 的 WSL（Windows Subsystem for Linux）环境中。WSL 提供了较完整的 Linux 用户空间环境，支持 Python、curses 和 TCP 网络编程，因此客户端可以直接在 WSL 终端中运行，并通过公网域名

连接到服务器。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/Linux_Class (code)# python3 server.py --host 0.0.0.0 --port 8765 --file shared.py
Collaborative server listening on 0.0.0.0:8765
Document: shared.py
JOIN client=glance address=('36.156.65.4', 2279)
JOIN client=sputnik address=('182.96.176.182', 3833)
OP v1 client=sputnik insert pos=70 char='\n' base=0
OP v2 client=sputnik insert pos=71 char='I' base=1
OP v3 client=sputnik insert pos=72 char=' ' base=2
OP v4 client=sputnik insert pos=73 char='a' base=3
OP v5 client=sputnik insert pos=74 char='m' base=4
OP v6 client=sputnik insert pos=75 char=' ' base=5
OP v7 client=sputnik insert pos=76 char='S' base=6
OP v8 client=sputnik insert pos=77 char='p' base=7
OP v9 client=sputnik insert pos=78 char='u' base=8
OP v10 client=sputnik insert pos=79 char='t' base=9
OP v11 client=sputnik insert pos=80 char='n' base=10
OP v12 client=sputnik insert pos=81 char='i' base=11
OP v13 client=sputnik insert pos=82 char='k' base=12
OP v14 client=glance insert pos=83 char=',' base=13
OP v15 client=glance insert pos=84 char=' ' base=14
OP v16 client=glance insert pos=85 char='a' base=15
OP v17 client=glance insert pos=86 char='n' base=16
OP v18 client=glance insert pos=87 char='d' base=17
OP v19 client=glance insert pos=88 char=' ' base=18
OP v20 client=glance insert pos=89 char='i' base=19
OP v21 client=glance delete pos=89 base=20
```

图 1 服务端运行界面

图 2 两个客户端连接服务端

前面三张运行截图展示了 TermSync 的基本启动和连接流程。第一张图中，服务端

已经在 Linux 服务器上启动并监听指定端口，终端输出中可以看到服务端正在等待客户端接入。后两张图分别展示两个不同客户端通过域名连接到同一个服务端实例，并进入终端编辑界面；这说明服务端可以同时维护多个客户端连接，新加入的客户端也能够获得当前共享文档状态，为后续的同步编辑、广播更新和保存验证提供了运行基础。

2 项目背景

多人协作编辑是一个典型的分布式一致性问题。多个用户可能在同一时间对同一份文本进行修改，如果系统只按照客户端本地光标位置直接修改文档，很容易出现字符位置错乱、删除目标错误、不同客户端内容不一致等问题。例如初始文本为 `hello world`，一个用户在位置 5 连续插入多个字符，另一个用户仍然基于旧版本请求删除位置 5 的字符。如果服务端不做额外处理，第二个用户想删除的字符和服务端当前文档中的位置就不再对应。

传统协作编辑系统常见的解决方案包括 OT 和 CRDT。本项目选择 OT：每个编辑操作携带自己基于的文档版本，服务端根据该版本之后已经发生的历史操作修正位置，最后将操作应用到当前文档。与更复杂的全客户端协同 OT 不同，本项目采用服务端主导方案，将全局排序和位置变换集中放在服务端，从而使实现更清晰，也更容易验证正确性。

本项目也具有明确的 Linux 课程背景。首先，服务端是一个典型的 TCP 长连接服务，涉及 socket 创建、地址绑定、监听、接受连接和消息收发。其次，每个客户端连接由独立线程处理，多个线程会同时访问共享文档，因此必须使用锁保护临界区。再次，自动保存和日志写入使用独立进程完成，主进程通过队列向工作进程发送快照、保存和日志事件，体现了进程间通信的使用。最后，文件加载、原子写入、备份文件、自动保存文件和编辑日志都体现了 Linux 文件系统编程的基本实践。

此外，终端编辑器本身也与 Linux 使用场景贴合。客户端基于 `curses` 运行在终端中，支持方向键移动、普通字符输入、回车换行、Backspace 删除、`Ctrl-S` 保存和 `Ctrl-Q` 退出。相比网页前端，这种实现更接近 Linux 命令行环境下的工具开发，也便于在服务器和 WSL 中演示。

从项目边界上看，TermSync 更偏向教学型协作编辑器。它重点展示字符级同步、网络通信、并发控制、文件持久化和自动化测试，而不追求语法高亮、代码补全、复杂撤销栈等完整 IDE 功能。这样的取舍使项目能够把主要篇幅集中在 Linux 程序设计课

程强调的系统调用、进程线程、终端交互和文件操作上。

3 功能描述

3.1 多人连接与文档同步

服务端通过 TCP socket 对外提供协作编辑服务。客户端连接后首先发送 `JOIN` 消息，服务端为其分配或确认 `client_id`，随后发送 `WELCOME` 和完整的 `DOC_STATE`。这样新加入的客户端可以立即得到当前文档内容、版本号和历史起始版本。

每个客户端连接由服务端中的一个独立线程处理。线程持续读取该客户端发来的 JSON Lines 消息，并根据消息类型分发到不同处理逻辑。编辑操作使用 `OP` 消息提交，保存请求使用 `SAVE` 消息提交，客户端也可以主动发送 `REQUEST_DOC` 请求完整文档。

3.2 字符级编辑操作

项目支持的核心编辑操作是字符级 `insert` 和 `delete`。插入操作包含插入位置和单个字符，删除操作包含删除位置。换行符 `\n` 被视为普通字符处理，因此多行文本可以自然表示为一个字符串。

客户端终端界面将用户按键转换为这些基础操作。普通可显示字符会生成 `insert`；回车会生成插入换行符；Backspace 会生成删除光标前一个字符的 `delete`。客户端本地光标会随着输入移动，但文档内容最终以服务端返回的 ACK 或远程广播为准。

这些操作在代码中的定义集中在代码仓库根目录下的 `document.py`：`VALID_KINDS` 限定操作类型只能是 `insert`、`delete` 和 `noop`，`Operation` 数据类保存 `kind`、`pos`、`char` 和可选的 `reason`。客户端在 `client.py` 的按键处理函数中把普通字符、回车和 Backspace 转换成 `Operation`，再由 `client_state.py` 封装成网络层的 `OP` 消息。服务端终端上看到的 `OP v...` 并不是另一种新的操作模型，而是 `server.py` 中日志格式化函数对服务端事件的可读化输出。

例如一条服务端输出 `OP v7 client=sputnik insert pos=76 char='S' base=6` 表示：服务端把这次操作确认为全局第 7 个版本，操作来自 `sputnik` 客户端，最终权威操作是在位置 76 插入字符 `S`，而客户端提交时基于的是版本 6。换行也按普通字符参与协同编辑，所以回车会显示为 `char='\n'`。

字符可以输入英文、数字、符号，也可以输入中文等宽字符。客户端使用 Python 的 `unicodedata` 模块，通过 `east_asian_width` 判断字符宽度，并提供显示列和字符

列之间的转换函数，确保光标移动和文本显示在终端中正确对齐。

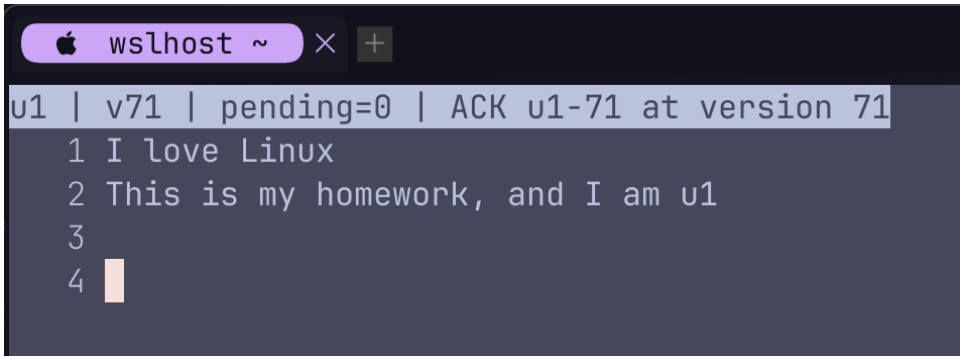


图 3 服务端英文输入

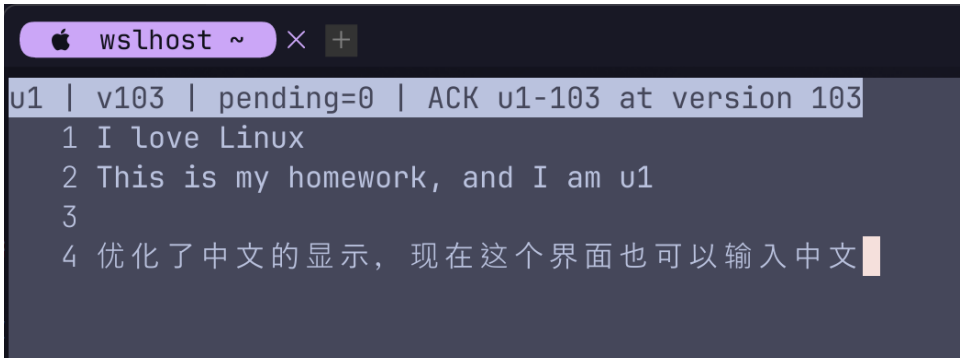


图 4 服务端中文输入

3.3 ACK 驱动的客户端发送队列

为了避免用户连续快速输入时丢失按键，客户端维护 `pending_queue`。每次用户输入产生的操作都会先进入队列。与此同时，客户端同一时刻最多只允许存在一个 `inflight_op`，也就是已经发送但尚未收到服务端 ACK 的操作。

当当前操作收到 ACK 后，客户端使用 ACK 中的权威操作更新本地文档，再从队列中取出下一个操作发送。这个设计的好处是客户端不需要实现复杂的 OT 逻辑，只需要维护本地队列和版本号；所有并发冲突都交给服务端处理。

3.4 服务端主导 OT 与全局版本

服务端的文档对象维护当前文本、当前版本号、有限历史记录和文档锁。所有编辑操作只有进入文档锁保护的临界区后，才会被分配新的 `server_version`。这个版本号既表示文档版本，也表示操作在全局历史中的顺序。

当客户端提交的 `base_version` 落后于服务端当前版本时，服务端会从历史记录中

取出该版本之后的所有操作，逐条对客户端操作进行位置变换。例如历史中已经发生过一次插入，那么位于插入点之后的操作位置需要右移；如果两个删除操作删除同一个字符，后到达的删除会被转换为 `noop`。

3.5 保存、自动保存与编辑日志

服务端启动时会加载指定文档文件。如果文件不存在，则以空文档开始。成功处理编辑操作后，服务端会把最新快照发送给自动保存进程。自动保存进程定期写入 `.autosave` 文件，避免每次按键都直接触发磁盘写入。

用户在客户端按下 `Ctrl-S` 后，客户端发送 `SAVE` 消息。服务端收到后将当前快照投递给自动保存进程，由工作进程写入正式文件。为了降低误覆盖风险，正式保存前会生成 `.bak` 备份文件。同时，服务端会记录加入、离开、操作、自动保存和手动保存等事件，形成 JSON Lines 格式的编辑日志。

图 5 服务端编辑日志与自动保存记录

图 5中展示了服务端运行目录中的保存产物和编辑日志。左侧可以看到正式文档 `shared.py`、自动保存文件 `shared.py.autosave`、备份文件 `shared.py.bak` 以及编辑日志 `shared.py.edit.log` 同时存在，说明服务端把文档内容、自动保存快照、覆盖前备份和事件记录分开管理。右侧打开的日志文件由多条 JSON 事件组成，每条记录包含 `event`、`path`、`timestamp` 和 `version` 等字段；其中 `autosave` 事件表示自动保存进程已经按固定间隔将当前快照写入 `.autosave` 文件。后续发生手动保存或编辑操作时，同一日志文件还会继续追加 `manual_save`、`operation` 等记录，用于追踪服务端文档状态的变化过程。

3.6 历史裁剪与重同步

为了避免服务端无限保存历史操作，文档对象使用有限长度的历史记录，默认保留最近 1000 条操作。服务端同时维护 `history_start_version`，表示当前历史仍然能够支持 OT 变换的最早基准版本。

如果某个客户端提交的 `base_version` 已经过旧，服务端无法可靠地根据有限历史进行变换，此时会返回 `RESYNC_REQUIRED`，并发送完整的 `DOC_STATE`。客户端收到后清空当前未确认操作，并请求或接受完整文档，从而恢复到服务端权威状态。

4 设计思路

4.1 总体架构

项目整体采用客户端–服务端架构。客户端只负责终端界面、键盘输入、发送队列和接收服务端消息；服务端负责网络连接管理、文档权威状态、操作排序、OT 变换、广播、保存和日志。这样划分后，系统中最关键的一致性逻辑集中在服务端，便于通过单元测试验证。

系统主要模块如下：

- `protocol.py`：封装 JSON Lines 消息的编码、解码和 socket 收发。
- `document.py`：实现权威文档、操作模型、OT 变换、版本历史和重同步判断。
- `server.py`：实现 TCP 服务端、客户端线程、消息分发、广播和自动保存进程管理。
- `client_state.py`：实现客户端同步状态、待发送队列、ACK 处理和远程操作应用。
- `client.py`：实现 curses 终端界面、光标移动、按键处理和网络连接。
- `autosave.py`：实现自动保存、手动保存、备份和日志写入工作进程。
- `tests/`：包含文档、客户端状态、服务端输出和自动保存相关单元测试。

4.2 网络协议设计

网络协议采用 JSON Lines，即每一条消息都是一个 JSON 对象，并以换行符结束。这种格式便于调试，也便于处理 TCP 粘包和拆包问题。接收端维护字节缓冲区，持续读取 socket 数据，直到遇到换行符后再解码一条完整消息。

典型的编辑操作消息包含消息类型、客户端标识、操作编号、基准版本和操作内容。例如：

```
{
    "type": "OP",
    "client_id": "u1",
    "op_id": "u1-12",
    "base_version": 8,
    "op": {
        "kind": "insert",
        "pos": 5,
        "char": "a"
    }
}
```

服务端处理成功后会给提交者返回 `ACK`，并向其他客户端广播 `REMOTE_OP`。这两类消息都携带服务端确认后的 `server_version` 和变换后的权威操作。客户端只应用服务端确认的操作，而不把本地预测结果直接作为最终文档。

4.3 文档锁与全局顺序

多客户端并发编辑时，服务端会有多个线程几乎同时收到 `OP` 消息。如果每个线程直接修改文档，就会出现竞争条件。因此文档对象内部使用可重入锁保护临界区。只有持有文档锁的线程才能检查版本、执行 OT、分配版本号、修改文本和写入历史。

这里的设计重点是：服务端不是按照客户端时间戳决定顺序，也不是按照网络包发出时间决定顺序，而是按照操作进入文档临界区的顺序决定全局顺序。进入临界区后分配的递增 `server_version` 就是最终权威顺序。这样可以避免分布式时间带来的不确定性。

另一方面，socket 发送和广播不能放在文档锁内部。如果某个客户端网络很慢，持锁发送会导致其他客户端的编辑操作无法进入临界区，扩大阻塞范围。因此服务端在文档锁内只生成 ACK 和广播消息，返回后再执行 socket I/O。

4.4 OT 位置变换规则

本项目使用字符级 OT，因此变换规则集中在位置调整上。当前操作会依次与它的 `base_version` 之后已经发生的历史操作进行比较：

- 历史操作是插入时，当前插入或删除如果位于该插入位置之后或相同位置，就需要右移一位。

- 历史操作是删除时，当前插入如果位于删除位置之后，需要左移一位。
- 历史操作是删除时，当前删除如果位于删除位置之后，需要左移一位。
- 两个删除操作删除同一位置时，后一个删除变为 `noop`，因为目标字符已经被删除。

服务端还会对最终操作做规范化处理，例如插入位置会被限制在文档长度范围内，越界删除会变为 `noop`。这能提高协议容错性，避免异常客户端破坏服务端文档状态。

4.5 客户端状态机

客户端状态机的核心是 `pending_queue` 和 `inflight_op`。用户输入不会直接阻塞等待网络响应，而是先进入队列；发送函数只在当前没有未确认操作时才取出队首并发送。每条操作发送时会记录当前客户端已知版本作为 `base_version`。

客户端收到 ACK 后，会检查 ACK 的 `op_id` 是否对应当前 `inflight_op`。如果匹配，就应用服务端返回的权威操作，更新版本号，清空 `inflight_op`，再继续发送队列中的下一个操作。客户端收到远程操作时，也会检查版本是否连续。如果发现远程版本跳跃，说明中间可能漏收消息，于是请求完整文档。

4.6 自动保存进程

自动保存和日志写入被放在独立进程中完成。主服务端进程处理网络和文档同步，自动保存进程负责磁盘 I/O。二者通过 `multiprocessing.Queue` 传递事件。服务端每处理完一次成功编辑，就发送最新快照；工作进程只保留最新内容，并按固定间隔写入 `.autosave` 文件。

手动保存时，服务端向队列发送 `SAVE_NOW` 事件。工作进程先复制旧文件作为 `.bak` 备份，再使用临时文件替换的方式写入正式文件。这样既体现了进程间通信，也减少了主服务端在网络请求处理过程中直接进行磁盘写入的时间。

5 测试与验证

5.1 单元测试

项目使用 Python 标准库 `unittest` 进行自动化测试。实际运行时，可以在 Linux 服务器克隆 `code` 分支后，进入克隆得到的代码仓库根目录执行测试命令：

```
python3 -m unittest discover -s tests -v
```

本地执行结果显示，共运行 17 个测试用例，所有测试均通过，输出结尾为 `Ran 17 tests ... OK`。

测试覆盖的关键场景包括：

- `server_version` 严格递增，保证服务端全局顺序明确。
- 基于旧版本的删除操作经过历史插入变换后，删除位置正确右移。
- 两个客户端删除同一字符时，后到达的删除被转换为 `noop`。
- 历史记录裁剪后，过旧 `base_version` 会触发重同步响应，并返回完整文档状态。
- 客户端连续输入时，只发送一个未确认操作，后续操作留在 `pending_queue` 中等待 ACK。
- 客户端发现远程版本不连续时，会发送 `REQUEST_DOC` 请求完整文档。
- 中文字符可以被插入，终端显示宽度辅助函数能正确处理宽字符。
- 手动保存时会在覆盖正式文件前生成备份文件。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/Linux_Class (code)# python3 -m unittest discover -s tests -v
test_manual_save_writes_backup (test_autosave.AutosaveTests.test_manual_save_writes_backup) ... ok
test_chinese_character_is_insertable (test_client_state.ClientSyncStateTests.test_chinese_character_is_insertable) ... ok
test_chinese_display_width_helpers (test_client_state.ClientSyncStateTests.test_chinese_display_width_helpers) ... ok
test_pending_queue_sends_next_only_after_ack (test_client_state.ClientSyncStateTests.test_pending_queue_sends_next_only_after_ack) ... ok
test_remote_gap_requests_document (test_client_state.ClientSyncStateTests.test_remote_gap_requests_document) ... ok
test_welcome_updates_server_assigned_client_id (test_client_state.ClientSyncStateTests.test_welcome_updates_server_assigned_client_id) ... ok
test_wide_character_backspace_queues_delete (test_client_state.ClientSyncStateTests.test_wide_character_backspace_queues_delete) ... ok
test_delete_position_transforms_after_prior_inserts (test_document.CollaborativeDocumentTests.test_delete_position_transforms_after_prior_ins
erts) ... ok
test_history_expiry_requests_resync (test_document.CollaborativeDocumentTests.test_history_expiry_requests_resync) ... ok
test_insert_chinese_character (test_document.CollaborativeDocumentTests.test_insert_chinese_character) ... ok
test_same_character_delete_becomes_noop (test_document.CollaborativeDocumentTests.test_same_character_delete_becomes_noop) ... ok
test_server_version_is_strictly_incremented (test_document.CollaborativeDocumentTests.test_server_version_is_strictly_incremented) ... ok
test_format_delete_operation_event (test_server.ServerOutputTests.test_format_delete_operation_event) ... ok
test_format_insert_operation_event (test_server.ServerOutputTests.test_format_insert_operation_event) ... ok
test_format_join_leave_events (test_server.ServerOutputTests.test_format_join_leave_events) ... ok
test_format_save_event (test_server.ServerOutputTests.test_format_save_event) ... ok
test_shutdown_ignores_non_owner_process (test_server.ServerOutputTests.test_shutdown_ignores_non_owner_process) ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.004s

OK
```

图 6 测试结果

5.2 本地验证

手动验证用于在不依赖公网、域名解析和安全组规则的情况下，先确认协作编辑器本身的核心功能是否正常。验证时在同一台 Linux 服务器上进行：一个终端运行服务端，同时使用 `tmux` 打开两个客户端窗口进行对比观察。首先在服务端终端中监听本机地址并指定协作编辑文件：

```
python3 server.py --host 127.0.0.1 --port 8765 --file shared.py
```

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/Linux_Class (code)# python3 server.py --host 127.0.0.1 --port 8765
 --file shared.py
Collaborative server listening on 127.0.0.1:8765
Document: shared.py
JOIN client=sputnik address=('127.0.0.1', 58768)
OP v1 client=sputnik delete pos=129 base=0
OP v2 client=sputnik delete pos=128 base=1
OP v3 client=sputnik delete pos=127 base=2
OP v4 client=sputnik delete pos=126 base=3
OP v5 client=sputnik delete pos=125 base=4
OP v6 client=sputnik delete pos=124 base=5
OP v7 client=sputnik delete pos=123 base=6
OP v8 client=sputnik delete pos=122 base=7
OP v9 client=sputnik delete pos=121 base=8
OP v10 client=sputnik delete pos=120 base=9
OP v11 client=sputnik delete pos=119 base=10
JOIN client=glance address=('127.0.0.1', 46514)
OP v12 client=glance insert pos=119 char='\n' base=11
OP v13 client=glance insert pos=120 char='\n' base=12
OP v14 client=glance insert pos=121 char='a' base=13
OP v15 client=glance insert pos=122 char='d' base=14
OP v16 client=glance insert pos=123 char='a' base=15
OP v17 client=glance insert pos=124 char='s' base=16
OP v18 client=glance insert pos=125 char='d' base=17
```

图 7 本地运行服务端界面

从服务端终端输出也可以看到，两个客户端加入时都会打印 `JOIN` 事件，并且地址字段中的 IP 均为 `127.0.0.1`。这说明 `sputnik` 和 `glance` 两个客户端虽然分别运行在不同的 `tmux` 窗口中，但网络连接都通过本机回环接口进入同一个服务端进程，符合本地手动验证的设计。

如果启动时出现 `Address already in use`，说明 `8765` 端口已经被已有进程占用，通常是前一次启动的服务端仍在运行。此时可以直接使用已有服务端继续启动客户端进行验证；如果需要重新启动服务端，则应先停止原服务端进程，或者临时换用其他端口并让两个客户端使用相同的新端口连接。

随后在 `tmux` 中打开两个客户端窗口或窗格，两个客户端都连接同一个本机服务端，但使用不同的 `client_id`：

```
python3 client.py --host 127.0.0.1 --port 8765 --client-id sputnik
python3 client.py --host 127.0.0.1 --port 8765 --client-id glance
```

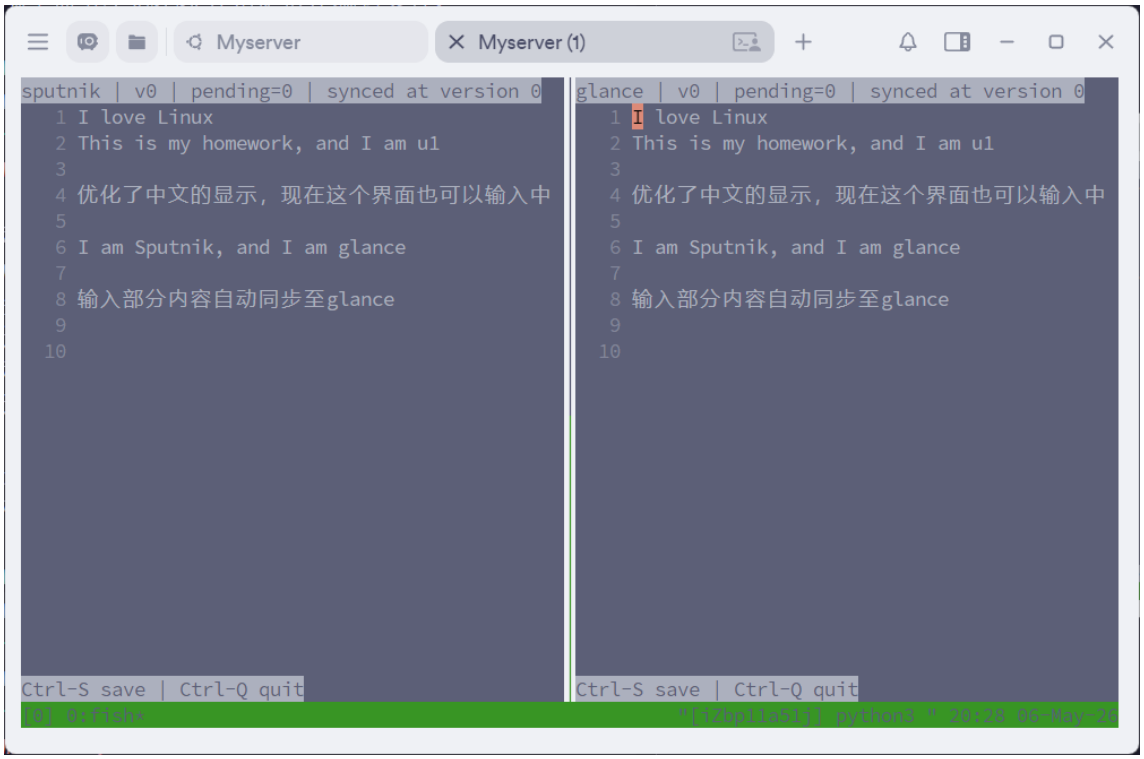


图 8 tmux 中两个本地客户端同步显示

图中左侧客户端使用 `sputnik`，右侧客户端使用 `glance`。两个客户端的状态栏都显示当前处于同步状态，文档内容也保持一致。实际验证时，在任意一个客户端输入英文、中文、换行或删除内容后，另一个客户端都会同步显示相同修改，说明本机环境下的消息发送、ACK 确认、服务端广播和客户端文档更新流程能够正常工作。

进一步验证异常情况时，可以在服务端终端中主动结束服务端进程。服务端关闭后，两个客户端不会继续显示为同步状态，而是在状态栏中显示 `error: connection closed`，提示与服务端的 TCP 连接已经断开；同时客户端中已经同步到的文档内容仍然保留在界面上，便于观察断线前的最终状态。

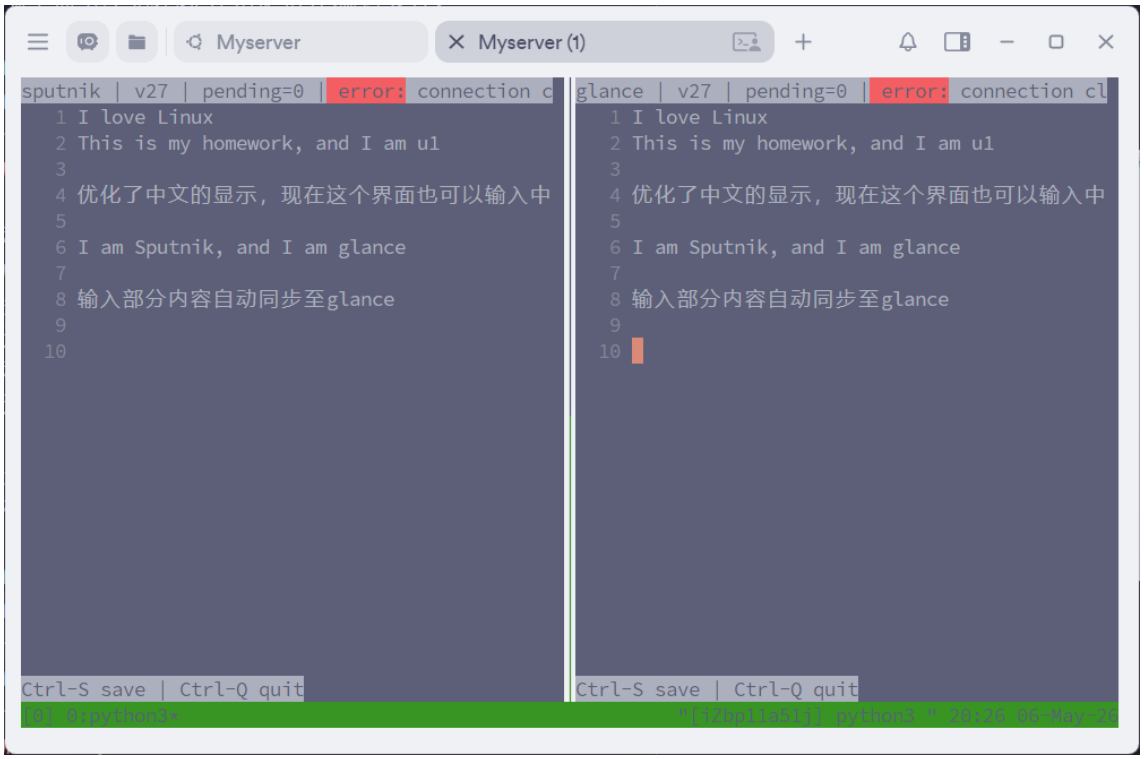


图 9 服务端关闭后客户端显示连接断开

5.3 部署验证

部署验证主要围绕真实公网运行流程展开。由于服务端运行在阿里云 ECS 服务器上，要让外部客户端能够访问该协作编辑服务，需要先在阿里云安全组中开放服务端使用的 8765 端口。只有安全组规则允许公网访问后，客户端才能通过 `glance02.xyz:8765` 连接到服务端。

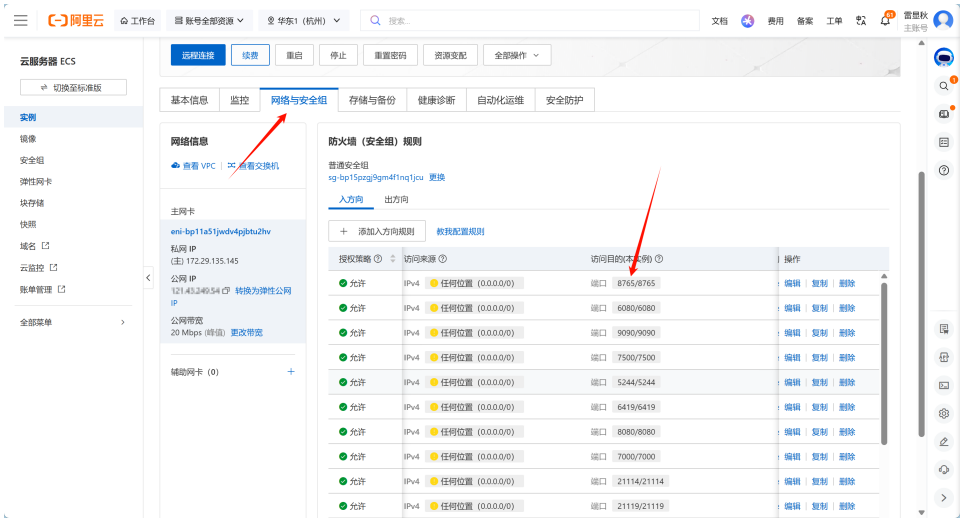


图 10 阿里云开放 8765 端口

完成端口放行后，在 Linux 服务器上启动服务端，监听 `0.0.0.0:8765`，并指定协作编辑文件。由于服务器已经绑定域名 `glance02.xyz`，客户端可以直接通过该域名连接：

```
python3 client.py --host glance02.xyz --port 8765 --client-id u1
```

随后在不同终端中启动两个客户端，分别使用不同 `client_id`。当其中一个客户端输入字符或换行时，另一个客户端会收到 `REMOTE_OP` 并同步显示更新。服务端终端会输出类似 `JOIN`、`OP v...`、`SAVE requested`、`LEAVE` 的事件日志，便于观察连接、编辑和保存流程。

下面是客户端 `sputnik` 连续输入换行和 `I am Sput` 时，服务端终端打印出的操作序列。可以看到每一次按键都会形成一条字符级 `insert` 操作，`v1` 到 `v10` 是服务端分配的全局递增版本号，`base` 则是客户端发送该操作时已确认的版本号。

```
OP v1 client=sputnik insert pos=70 char='\n' base=0
OP v2 client=sputnik insert pos=71 char='I' base=1
OP v3 client=sputnik insert pos=72 char=' ' base=2
OP v4 client=sputnik insert pos=73 char='a' base=3
OP v5 client=sputnik insert pos=74 char='m' base=4
OP v6 client=sputnik insert pos=75 char=' ' base=5
OP v7 client=sputnik insert pos=76 char='S' base=6
OP v8 client=sputnik insert pos=77 char='p' base=7
OP v9 client=sputnik insert pos=78 char='u' base=8
OP v10 client=sputnik insert pos=79 char='t' base=9
```

这段日志能够直接反映 ACK 驱动发送队列的效果：客户端收到上一条操作的 ACK 后才继续发送下一条，所以 `base` 会随着服务端版本逐步递增。另一方面，`pos` 也连续增加，说明服务端按字符位置把换行和后续英文字符顺序应用到了同一份权威文档中。

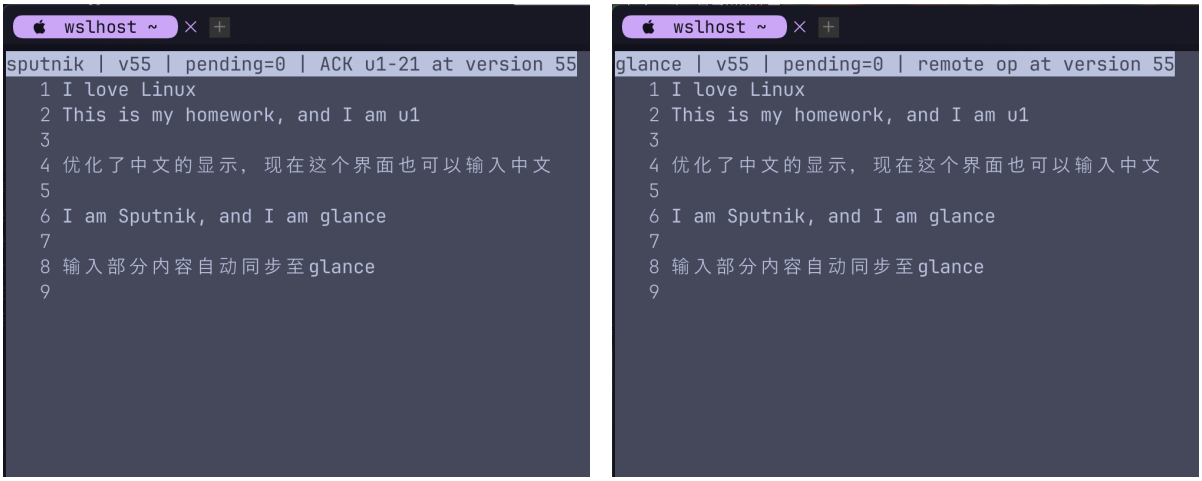


图 11 客户端 sputnik 输入内容后客户端 glance 自动同步显示

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/Linux_Class (code)# python3 server.py --host 0.0.0.0 --port 8765 --file shared.py
Collaborative server listening on 0.0.0.0:8765
Document: shared.py
JOIN client=sputnik address=('182.96.176.182', 2291)
JOIN client=glance address=('36.156.65.4', 15875)
OP v1 client=sputnik insert pos=120 char='\n' base=0
OP v2 client=sputnik insert pos=121 char='测' base=1
OP v3 client=sputnik insert pos=122 char='试' base=2
OP v4 client=sputnik insert pos=123 char='部' base=3
OP v5 client=sputnik insert pos=124 char='分' base=4
LEAVE client=sputnik
LEAVE client=glance
^CSAVE shutdown version=5 file=shared.py
```

图 12 服务端终端输出 JOIN、OP、SAVE、LEAVE 等事件日志

为了验证 OT 行为，可以构造两个客户端基于不同版本编辑同一位置的场景。例如一个客户端连续在某个位置插入字符，另一个客户端基于旧版本删除该位置字符。根据服务端日志中的 `transform_trace`，可以观察到操作位置在应用前被转换。最终两个客户端显示的文档内容保持一致。

```
Vaults  SFTP  Myserver
root@iZbp11a51jwdv4pjbv2r1oZ ~/Linux_Class (code)# tail -n 50 shared.py.edit.log | jq 'select(.op_id=="stale-transform")'

{
  "base_version": 0,
  "client_id": "stale",
  "content_length": 130,
  "event": "operation",
  "op_id": "stale-transform",
  "original_op": {
    "char": "旧",
    "kind": "insert",
    "pos": 120
  },
  "server_version": 5,
  "timestamp": 1777911047.3625264,
  "transform_trace": [
    {
      "after": {
        "char": "旧",
        "kind": "insert",
        "pos": 121
      },
      "against_op": {
        "char": "旧",
        "kind": "insert",
        "pos": 120
      },
      "against_version": 1,
      "before": {
        "char": "旧",
        "kind": "insert",
        "pos": 120
```

图 13 编辑日志中的 transform_trace 位置变换记录

图中将原始 JSON 日志中的 `transform_trace` 提取为位置变换过程。该操作的 `op_id` 是 `stale-transform`，提交时基于旧版本 0，并在服务端版本 5 被确认。操作原本希望在位置 120 插入字符，服务端根据之后已经发生的历史插入操作依次调整位置，最终将其应用到位置 124。

表 1 transform_trace 位置变换摘要

against_version	against_op	before_pos → after_pos
1	insert(pos=120, char=旧)	120 → 121
2	insert(pos=121, char=A)	121 → 122
3	insert(pos=122, char=B)	122 → 123
4	insert(pos=123, char=C)	123 → 124

该验证说明项目不仅可以在本机 127.0.0.1 上运行，也可以在真实网络环境中作为远程协作工具使用。

6 问题与解决方案

6.1 并发操作顺序不确定

问题：多个客户端线程可能同时提交编辑操作。如果服务端没有统一排序，不同线程对文档的修改顺序可能不稳定，客户端最终内容也可能不一致。

解决方案：将权威文档状态集中在服务端，并在 CollaborativeDocument 内部使用文档锁。每个操作进入锁保护的临界区后才分配 server_version。因此全局顺序由服务端临界区顺序决定，而不是由客户端时间或网络延迟决定。

6.2 旧版本操作位置失效

问题：客户端提交操作时使用的是本地版本。如果该操作到达服务端时，其他客户端已经修改过文档，那么原始位置可能已经不再正确。

解决方案：服务端保存有限历史。处理旧版本操作时，从 base_version 之后的历史操作开始逐条执行 OT 位置变换，将操作转换到当前版本后再应用。这样旧版本操作仍然能表达用户原本想编辑的位置。

6.3 连续输入容易丢失或乱序

问题：用户在终端中快速输入多个字符时，如果客户端每次按键都立即发送，同时又没有处理未确认操作，容易造成本地版本混乱。

解决方案：客户端使用 pending_queue 保存所有输入，并限制同一时刻最多一个 inflight_op。只有收到当前操作的 ACK 后，才发送下一个操作。这样牺牲了一点发

送并行度，但显著降低了客户端同步复杂度。

6.4 慢客户端可能阻塞服务端

问题：如果服务端在文档锁内向所有客户端广播消息，而某个客户端网络很慢，socket 发送可能阻塞，从而导致其他客户端无法继续提交编辑操作。

解决方案：文档锁内只完成状态修改和消息构造，ACK 和广播都放在锁外执行。广播时也只短暂持有客户端列表锁来复制收件人列表，真正发送时不再持有该锁。

6.5 历史无限增长

问题：如果服务端永久保存所有操作历史，长时间运行后内存占用会持续增加。

解决方案：历史记录使用有限长度队列保存最近的操作，最大条数由代码中的历史限制参数控制。同时维护 `history_start_version`，记录当前历史能够支持的最早基准版本。当客户端版本太旧，已经无法根据保留历史进行 OT 时，服务端返回 `RESYNC_REQUIRED` 并发送完整文档。

6.6 文件保存可能覆盖旧内容

问题：手动保存会覆盖正式文件，如果误操作或程序异常，可能丢失旧版本内容。

解决方案：保存前生成 `.bak` 备份文件，并使用临时文件替换的方式进行原子写入。自动保存则写入独立的 `.autosave` 文件，不直接覆盖正式文件。

6.7 中文字符显示宽度

问题：终端中中文字符通常占两个显示列，如果简单按照字符串长度移动光标，显示位置会错乱。

解决方案：客户端使用 Python 的 `unicodedata` 模块，通过 `east_asian_width` 判断宽字符，并提供显示列和字符列之间的转换函数。测试中也包含中文字符插入和显示宽度计算场景。

7 项目不足与改进方向

虽然 TermSync 已经能够完成多人连接、实时同步、保存、自动保存、日志记录和测试验证，但它仍然是一个课程项目规模的协作编辑器，和成熟协作工具相比还存在

一些不足。首先，当前编辑粒度是字符级操作，便于说明 OT 原理，但快速输入大量文本时会产生较多网络消息；后续可以考虑把连续输入合并为批量操作。其次，系统暂时没有加入用户认证和权限控制，只适合可信网络或课堂演示场景，如果要公开部署，需要补充连接鉴权、访问控制和异常客户端隔离。再次，客户端的终端界面功能较基础，还可以继续增加状态栏、只读提示、断线重连和冲突恢复提示，使使用体验更加完整。

从实现角度看，后续还可以继续扩展测试范围。例如增加更复杂的并发插入、插入与删除交错、历史裁剪后客户端重连等集成测试；在服务端加入更细粒度的运行日志和错误统计；在保存模块中加入异常注入测试，验证磁盘写入失败时备份文件和自动保存文件是否仍然可靠。这些改进能够进一步提升项目的稳定性，也能让课程报告中的系统设计更加接近真实 Linux 服务程序。

8 代码展示

本节展示的文件名均以服务器克隆 `code` 分支后得到的代码仓库根目录为基准。报告工程中的 `\inputminted` 路径只用于排版时引入代码片段，不代表服务器上的运行目录结构。

8.1 JSON Lines 协议封装

协议层负责把 Python 字典编码为 UTF-8 JSON Lines 数据帧，并在接收端按换行符拆分消息。发送函数内部使用锁，保证同一连接上多线程发送时不会出现字节交叉。

```python
import json
import socket
import threading
from typing import Any, Dict


Message = Dict[str, Any]
MAX_LINE_BYTES = 1024 * 1024


def encode_message(message: Message) -> bytes:
    """ 把一条 JSON 消息编码为 UTF-8 的 JSON Lines 数据帧。"""
    return (
        json.dumps(message, ensure_ascii=False, separators=(",",
        ↪  ":")).encode("utf-8")
        + b"\n"
    )
```

```python


def decode_message(line: bytes) -> Message:
    """ 把一条 JSON Lines 数据帧解码为消息字典。 """
    if not line:
        raise EOFError("connection closed")
    try:
        payload = json.loads(line.decode("utf-8"))
    except json.JSONDecodeError as exc:
        raise ValueError(f"invalid JSON message: {exc}") from exc
    if not isinstance(payload, dict):
        raise ValueError("protocol message must be a JSON object")
    return payload


class JsonLineSocket:
    """ 对套接字做一层轻量封装，用于收发按换行分隔的 JSON 消息。 """

    def __init__(self, sock: socket.socket):
        self.sock = sock
        self._buffer = bytearray()
        self._send_lock = threading.Lock()

    def recv_message(self) -> Message:
        while True:
            newline_index = self._buffer.find(b"\n")
            if newline_index >= 0:
                line = bytes(self._buffer[:newline_index])
                del self._buffer[: newline_index + 1]
                return decode_message(line)

            chunk = self.sock.recv(4096)
            if not chunk:
                raise EOFError("connection closed")
            self._buffer.extend(chunk)
            if len(self._buffer) > MAX_LINE_BYTES:
                raise ValueError("protocol message exceeds maximum line length")

    def send_message(self, message: Message) -> None:
        data = encode_message(message)
        # 同一个客户端连接可能被读线程、UI 线程或服务端广播逻辑同时使用。
        # 这里用发送锁保证一条 JSON Lines 消息会完整写入套接字，
        # 避免两次 sendall 的字节交叉在一起，导致对端 JSON 解析失败。
        with self._send_lock:
            self.sock.sendall(data)
```

8.2 编辑操作模型

编辑操作被抽象为 `Operation`，支持 `insert`、`delete` 和 `noop`。从网络消息解析操作时会进行类型检查，例如插入操作必须携带单个字符。

```python
from collections import deque
from dataclasses import dataclass, field
from threading import RLock
from typing import Any, Deque, Dict, Iterable, List, Optional


VALID_KINDS = {"insert", "delete", "noop"}


@dataclass
class Operation:
    kind: str
    pos: int = 0
    char: Optional[str] = None
    reason: Optional[str] = None

    @classmethod
    def from_dict(cls, payload: Dict[str, Any]) -> "Operation":
        if not isinstance(payload, dict):
            raise ValueError("operation must be a JSON object")
        kind = payload.get("kind")
        if kind not in VALID_KINDS:
            raise ValueError(f"unsupported operation kind: {kind!r}")
        pos = payload.get("pos", 0)
        if not isinstance(pos, int):
            raise ValueError("operation pos must be an integer")
        char = payload.get("char")
        if kind == "insert":
            if not isinstance(char, str) or len(char) != 1:
                raise ValueError("insert operation requires a single-character char")
        else:
            char = None
        reason = payload.get("reason")
        if reason is not None and not isinstance(reason, str):
            reason = str(reason)
        return cls(kind=kind, pos=pos, char=char, reason=reason)

    def clone(self) -> "Operation":
        return Operation(self.kind, self.pos, self.char, self.reason)

    def to_dict(self) -> Dict[str, Any]:
```

```
        payload: Dict[str, Any] = {"kind": self.kind, "pos": self.pos}
        if self.char is not None:
            payload["char"] = self.char
        if self.reason:
            payload["reason"] = self.reason
        return payload

    def as_noop(self, reason: str) -> "Operation":
        return Operation("noop", self.pos, None, reason)
```

8.3 服务端 OP 日志输出

服务端终端中的 `OP v...` 输出由 `format_operation_event` 生成。它从操作日志事件中取出服务端版本、客户端编号、变换后的操作、字符内容和基准版本，并格式化成便于演示观察的一行文本。

```
def format_operation_event(entry: Message) -> str:
    op = entry.get("transformed_op", {})
    kind = op.get("kind", "unknown") if isinstance(op, dict) else "unknown"
    pos = op.get("pos", "?") if isinstance(op, dict) else "?"
    message = (
        f"OP v{entry.get('server_version')} client={entry.get('client_id')} "
        f"{kind} pos={pos}"
    )
    if isinstance(op, dict) and kind == "insert":
        message += f" char={op.get('char')!r}"
    message += f" base={entry.get('base_version')}"
    return message
```

8.4 OT 位置变换规则

`transform_against` 是 OT 核心函数。它根据一条已经发生的历史操作，调整当前操作的位置，或者在重复删除同一字符时将操作转换为 `noop`。

```
def transform_against(op: Operation, previous: Operation) -> Operation:
    """ 根据一条已经发生的历史操作，变换当前操作的位置。"""
    result = op.clone()
    if result.kind == "noop" or previous.kind == "noop":
        return result

    if previous.kind == "insert":
        # 历史中已经发生过一次插入，会把它后面的字符整体向右推一格。
        # 因此当前操作如果作用在插入点之后，或者正好作用在同一位置，
        # 都需要把位置加 1，才能继续指向原来想编辑的字符。
```

```
        if result.kind == "insert" and previous.pos <= result.pos:
            result.pos += 1
        elif result.kind == "delete" and previous.pos <= result.pos:
            result.pos += 1
        return result

    if previous.kind == "delete":
        # 历史中已经发生过一次删除，会把它后面的字符整体向左拉一格。
        # 插入操作只在历史删除点严格位于自己之前时左移；如果同位置插入，
        # 插入仍然发生在这个位置。删除操作如果删除同一个字符，则变成 noop。
        if result.kind == "insert" and previous.pos < result.pos:
            result.pos -= 1
        elif result.kind == "delete":
            if previous.pos < result.pos:
                result.pos -= 1
            elif previous.pos == result.pos:
                return result.as_noop("same character already deleted")
        return result

    raise ValueError(f"unsupported history operation kind: {previous.kind!r}")
```

8.5 服务端权威文档处理流程

`process_operation` 展示了服务端主导 OT 的完整流程：解析操作、检查版本、遍历历史、执行变换、规范化操作、分配版本号、应用文本修改、写入历史并生成 ACK 和广播消息。

```
    def process_operation(
        self,
        client_id: str,
        op_id: str,
        base_version: int,
        op_payload: Dict[str, Any],
    ) -> ProcessResult:
        try:
            original_op = Operation.from_dict(op_payload)
        except ValueError as exc:
            return ProcessResult(
                status="error",
                messages=[{"type": "ERROR", "op_id": op_id, "message": str(exc)}],
            )

        with self._lock:
            # 下面这段就是服务端的“文档处理临界区”：
            # 1. 检查客户端版本是否还能被历史覆盖；
```

```
        # 2. 对旧版本操作做 OT 位置变换；
        # 3. 分配新的 server_version；
        # 4. 应用操作并写入有限历史。
        # 套接字发送不在这里做，避免慢客户端长期占用文档锁。
        if not isinstance(base_version, int):
            return ProcessResult(
                status="error",
                messages=[
                    {
                        "type": "ERROR",
                        "op_id": op_id,
                        "message": "base_version must be an integer",
                    }
                ],
            )

        if base_version < self.history_start_version or base_version >
        ↪  self.version:
            return ProcessResult(
                status="resync",
                messages=[
                    {
                        "type": "RESYNC_REQUIRED",
                        "op_id": op_id,
                        "reason": "operation history expired"
                        if base_version < self.history_start_version
                        else "client version is ahead of server",
                        "server_version": self.version,
                        "history_start_version": self.history_start_version,
                    },
                    self._snapshot_unlocked(),
                ],
            )

        transformed = original_op.clone()
        transform_trace: List[Dict[str, Any]] = []
        # 客户端提交的 base_version 可能落后于服务端当前版本。
        # 这里把该版本之后的每一条权威历史操作依次拿出来，
        # 将客户端操作一步步变换到“现在”可以安全应用的位置。
        for entry in self._history_after_unlocked(base_version):
            before = transformed.to_dict()
            transformed = transform_against(transformed, entry.op)
            after = transformed.to_dict()
            if before != after:
                transform_trace.append(
                    {
                        "against_version": entry.server_version,
```

```
                        "against_op": entry.op.to_dict(),
                        "before": before,
                        "after": after,
                    }
                )

        transformed = self._normalize_for_apply_unlocked(transformed)
        # 操作只有进入临界区并完成 OT 后才获得 server_version。
        # 这个递增编号就是全局操作顺序，不依赖客户端时间戳或网络发送顺序。
        self.version += 1
        server_version = self.version
        self._text = apply_operation_to_text(self._text, transformed)

        history_entry = HistoryEntry(
            server_version=server_version,
            client_id=client_id,
            op_id=op_id,
            base_version=base_version,
            original_op=original_op,
            op=transformed,
        )
        self.history.append(history_entry)
        self._refresh_history_start_unlocked()

        ack = {
            "type": "ACK",
            "client_id": client_id,
            "op_id": op_id,
            "base_version": base_version,
            "server_version": server_version,
            "version": self.version,
            "op": transformed.to_dict(),
            "original_op": original_op.to_dict(),
        }
        remote = {
            "type": "REMOTE_OP",
            "client_id": client_id,
            "op_id": op_id,
            "server_version": server_version,
            "version": self.version,
            "op": transformed.to_dict(),
        }
        log_event = {
            "event": "operation",
            "server_version": server_version,
            "client_id": client_id,
            "op_id": op_id,
```

```
                "base_version": base_version,
                "original_op": original_op.to_dict(),
                "transformed_op": transformed.to_dict(),
                "transform_trace": transform_trace,
                "content_length": len(self._text),
            }
            return ProcessResult(
                status="ok",
                ack=ack,
                remote=remote,
                log_event=log_event,
                snapshot_content=self._text,
                snapshot_version=self.version,
            )
```

8.6 服务端锁外发送与广播

服务端处理操作时，文档修改已经在 `process_operation` 内完成。返回结果后，服务端再发送 ACK 和广播远程操作，从而避免慢客户端阻塞文档锁。

```
    def _handle_operation(self, client: ClientConnection, message: Message) -> None:
        # process_operation 内部会获取文档锁，并在临界区内完成 OT、
        # server_version 分配和文档更新。函数返回后，文档锁已经释放。
        result = self.document.process_operation(
            client_id=client.client_id,
            op_id=str(message.get("op_id", "")),
            base_version=message.get("base_version", -1),
            op_payload=message.get("op", {}),
        )

        if result.status == "ok":
            self._record_successful_operation(result)
            # 注意：ACK 和 REMOTE_OP 都在文档锁外发送。
            # 这样即使某个客户端网络很慢，也不会阻塞其他线程进入文档临界区。
            if result.ack is not None:
                client.send(result.ack)
            if result.remote is not None:
                self._broadcast(result.remote, exclude_client_id=client.client_id)
            return

        for response in result.messages:
            client.send(response)

    def _record_successful_operation(self, result: ProcessResult) -> None:
        if result.snapshot_content is not None and result.snapshot_version is not
        ↪ None:
```

```
            self._autosave_queue.put(
                {
                    "type": "SNAPSHOT",
                    "content": result.snapshot_content,
                    "version": result.snapshot_version,
                }
            )
        if result.log_event is not None:
            self._log(result.log_event)

    def _choose_client_id(self, requested: object) -> str:
        with self.clients_lock:
            if isinstance(requested, str) and requested and requested not in
            ↪ self.clients:
                return requested
            while True:
                candidate = f"u{self._next_client_number}"
                self._next_client_number += 1
                if candidate not in self.clients:
                    return candidate

    def _register_client(self, client: ClientConnection) -> None:
        with self.clients_lock:
            self.clients[client.client_id] = client

    def _unregister_client(self, client_id: str) -> None:
        with self.clients_lock:
            self.clients.pop(client_id, None)

    def _broadcast(self, message: Message, exclude_client_id: Optional[str] = None)
    ↪ -> None:
        # 持有客户端注册表锁时只复制收件人列表，释放锁之后再执行套接字 I/O。
        # 这里的锁只保护 clients 字典，不保护文档内容。复制收件人列表后立刻释放，
        # 可以避免广播阶段和客户端加入/退出管理互相长时间阻塞。
        with self.clients_lock:
            recipients: List[ClientConnection] = [
                client
                for cid, client in self.clients.items()
                if cid != exclude_client_id
            ]
        failed: List[str] = []
        for recipient in recipients:
            if not self._safe_send(recipient, message):
                failed.append(recipient.client_id)
        for client_id in failed:
            self._unregister_client(client_id)

```

```python
    def _safe_send(self, client: ClientConnection, message: Message) -> bool:
        try:
            client.send(message)
            return True
        except OSError:
            return False
```

8.7 客户端发送队列与 ACK 处理

客户端同步状态使用 `pending_queue` 和 `inflight_op` 控制发送节奏。它保证连续按键不会丢失，同时把最终文档更新交给服务端 ACK 或远程操作决定。

```python
@dataclass
class PendingOperation:
    op_id: str
    op: Operation
    base_version: Optional[int] = None


class ClientSyncState:
    """ 缓存本地输入，并保证同一时刻最多只有一个未确认操作。"""

    def __init__(self, client_id: str, send_message: Callable[[Message], None]):
        self.client_id = client_id
        self.send_message = send_message
        self.content = ""
        self.version = 0
        self.history_start_version = 0
        self.pending_queue: Deque[PendingOperation] = deque()
        self.inflight_op: Optional[PendingOperation] = None
        self._next_op_number = 1
        self.status = "disconnected"

    def set_document(self, content: str, version: int, history_start_version: int = 0) -> None:
        self.content = content
        self.version = version
        self.history_start_version = history_start_version
        self.inflight_op = None
        self.status = f"synced at version {version}"
        self._pump()

    def queue_operation(self, op: Operation) -> str:
        op_id = f"{self.client_id}-{self._next_op_number}"
        self._next_op_number += 1
```

```python
        # 用户连续按键时不能因为上一个操作还没 ACK 就丢输入。
        # 所有本地输入先进入 pending_queue，再由 _pump 按顺序发送。
        self.pending_queue.append(PendingOperation(op_id=op_id, op=op))
        self.status = f"queued {len(self.pending_queue)} operation(s)"
        self._pump()
        return op_id

    def handle_message(self, message: Message) -> None:
        message_type = message.get("type")
        if message_type == "DOC_STATE":
            self.set_document(
                content=str(message.get("content", "")),
                version=int(message.get("version", 0)),
                history_start_version=int(message.get("history_start_version", 0)),
            )
        elif message_type == "WELCOME":
            self.client_id = str(message.get("client_id", self.client_id))
            self.status = f"connected as {self.client_id}"
        elif message_type == "ACK":
            self._handle_ack(message)
        elif message_type == "REMOTE_OP":
            self._handle_remote_op(message)
        elif message_type == "RESYNC_REQUIRED":
            self.status = "server requested resync"
            self.inflight_op = None
            self.send_message({"type": "REQUEST_DOC"})
        elif message_type == "SAVE_QUEUED":
            self.status = f"save queued at version {message.get('version')}"
        elif message_type == "ERROR":
            self.status = f"error: {message.get('message')}"

    def _handle_ack(self, message: Message) -> None:
        op_id = str(message.get("op_id", ""))
        if self.inflight_op is None or self.inflight_op.op_id != op_id:
            self.status = f"ignored stale ACK {op_id}"
            return
        # 客户端不自行决定本地操作最终落点，而是等待服务端 ACK 中的
        # 权威 op。这个 op 可能已经被服务端 OT 变换过。
        self._apply_authoritative_op(message)
        self.inflight_op = None
        self.status = f"ACK {op_id} at version {self.version}"
        self._pump()

    def _handle_remote_op(self, message: Message) -> None:
        incoming_version = int(message.get("version", 0))
        if incoming_version <= self.version:
            return
```

```python
        if incoming_version != self.version + 1:
            self.status = "missed remote operation; requesting document"
            self.send_message({"type": "REQUEST_DOC"})
            return
        self._apply_authoritative_op(message)
        self.status = f"remote op at version {self.version}"

    def _apply_authoritative_op(self, message: Message) -> None:
        op = Operation.from_dict(message.get("op", {"kind": "noop"}))
        self.content = apply_operation_to_text(self.content, op)
        self.version = int(message.get("version", self.version))

    def _pump(self) -> None:
        if self.inflight_op is not None or not self.pending_queue:
            return
        next_op = self.pending_queue.popleft()
        next_op.base_version = self.version
        self.inflight_op = next_op
        # 每个客户端同一时刻只发送一个未确认操作。
        # 这样客户端侧不需要实现复杂 OT，只要记住当前版本并等待 ACK。
        self.send_message(
            {
                "type": "OP",
                "client_id": self.client_id,
                "op_id": next_op.op_id,
                "base_version": next_op.base_version,
                "op": next_op.op.to_dict(),
            }
        )
```

8.8 终端按键处理

终端客户端将用户按键映射为协作编辑操作。普通字符、回车和 Backspace 分别转化为插入字符、插入换行和删除光标前字符；`Ctrl-S` 和 `Ctrl-Q` 分别用于保存和退出。

```python
    def _handle_key(self, key: KeyInput) -> None:
        if key in (CTRL_Q, CONTROL_Q):
            self.running = False
            return
        if key in (CTRL_S, CONTROL_S):
            self.transport.send_message({"type": "SAVE"})
            return
        if key in BACKSPACE_KEYS:
            self._delete_before_cursor()
```

```
            return
        if isinstance(key, str):
            if key == "\n":
                self.state.queue_operation(Operation("insert", self.cursor_pos, "\n"))
                self.cursor_pos += 1
                return
            if is_insertable_character(key):
                self.state.queue_operation(Operation("insert", self.cursor_pos, key))
                self.cursor_pos += 1
            return
        if key == curses.KEY_LEFT:
            self.cursor_pos = max(0, self.cursor_pos - 1)
            return
        if key == curses.KEY_RIGHT:
            self.cursor_pos = min(len(self.state.content), self.cursor_pos + 1)
            return
        if key == curses.KEY_UP:
            line, col = pos_to_line_col(self.state.content, self.cursor_pos)
            current_line = split_lines_for_display(self.state.content)[line]
            display_col = text_display_width(current_line[:col])
            self.cursor_pos = line_col_to_pos_by_display(self.state.content, line - 1, display_col)
            return
        if key == curses.KEY_DOWN:
            line, col = pos_to_line_col(self.state.content, self.cursor_pos)
            current_line = split_lines_for_display(self.state.content)[line]
            display_col = text_display_width(current_line[:col])
            self.cursor_pos = line_col_to_pos_by_display(self.state.content, line + 1, display_col)
            return
        if key in (curses.KEY_BACKSPACE, 127, 8):
            self._delete_before_cursor()
            return
        if key in (curses.KEY_ENTER, 10, 13):
            self.state.queue_operation(Operation("insert", self.cursor_pos, "\n"))
            self.cursor_pos += 1

    def _delete_before_cursor(self) -> None:
        if self.cursor_pos > 0:
            delete_pos = self.cursor_pos - 1
            self.state.queue_operation(Operation("delete", delete_pos))
            self.cursor_pos = delete_pos
```

8.9 自动保存与备份

自动保存进程通过队列接收主服务端发来的事件。它负责定期写入自动保存文件，手动保存时生成备份并写入正式文件，同时追加 JSON 日志。

```python
def atomic_write_text(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp_path = path.with_name(path.name + ".tmp")
    tmp_path.write_text(content, encoding="utf-8")
    tmp_path.replace(path)


def write_with_backup(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if path.exists():
        backup_path = path.with_name(path.name + ".bak")
        shutil.copy2(path, backup_path)
    atomic_write_text(path, content)


def append_json_log(path: Path, entry: Dict[str, Any]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("a", encoding="utf-8") as handle:
        handle.write(json.dumps(entry, ensure_ascii=False, sort_keys=True) + "\n")


def autosave_worker(
    event_queue: Queue,
    document_path: str,
    log_path: str,
    interval_seconds: float = 5.0,
) -> None:
    signal.signal(signal.SIGINT, signal.SIG_IGN)
    # 自动保存和日志写入放在单独进程中完成。
    # 主服务端只需要通过 multiprocessing.Queue 投递事件，不必在处理
    # 网络请求或文档锁时直接做磁盘 I/O，从而体现进程间通信的使用。
    target_path = Path(document_path)
    autosave_path = target_path.with_name(target_path.name + ".autosave")
    log_file = Path(log_path)
    latest_content: Optional[str] = None
    latest_version = 0
    next_save = time.monotonic() + interval_seconds

    while True:
        timeout = max(0.0, next_save - time.monotonic())
        try:
```

```
            event = event_queue.get(timeout=timeout)
        except Empty:
            event = None

        if event:
            event_type = event.get("type")
            if event_type == "STOP":
                if latest_content is not None:
                    atomic_write_text(autosave_path, latest_content)
                return
            if event_type == "SNAPSHOT":
                # 服务端每处理完一次成功编辑，就发送最新快照。
                # 工作进程只保存最后一次快照，定时落盘，避免每个按键都写文件。
                latest_content = event.get("content", "")
                latest_version = int(event.get("version", latest_version))
            elif event_type == "SAVE_NOW":
                # 手动保存写入正式文件，并在覆盖前生成 .bak 备份。
                content = event.get("content", latest_content or "")
                latest_content = content
                latest_version = int(event.get("version", latest_version))
                write_with_backup(target_path, content)
                append_json_log(
                    log_file,
                    {
                        "event": "manual_save",
                        "version": latest_version,
                        "path": str(target_path),
                        "timestamp": time.time(),
                    },
                )
            elif event_type == "LOG":
                entry = dict(event.get("entry", {}))
                entry.setdefault("timestamp", time.time())
                append_json_log(log_file, entry)

        if time.monotonic() >= next_save:
            if latest_content is not None:
                atomic_write_text(autosave_path, latest_content)
                append_json_log(
                    log_file,
                    {
                        "event": "autosave",
                        "version": latest_version,
                        "path": str(autosave_path),
                        "timestamp": time.time(),
                    },
                )
```

```
            next_save = time.monotonic() + interval_seconds
```

8.10 文档一致性测试

文档测试覆盖了 OT 中最关键的情况：旧版本删除位置经过多个插入后右移、重复删除变为 noop、历史过期触发重同步，以及版本号严格递增。

```
class CollaborativeDocumentTests(unittest.TestCase):
    def test_delete_position_transforms_after_prior_inserts(self):
        doc = CollaborativeDocument("hello world")
        for index, pos in enumerate([5, 6, 7, 8], start=1):
            result = doc.process_operation(
                client_id="A",
                op_id=f"A-{index}",
                base_version=index - 1,
                op_payload={"kind": "insert", "pos": pos, "char": "!"},
            )
            self.assertEqual(result.status, "ok")

        result = doc.process_operation(
            client_id="B",
            op_id="B-1",
            base_version=0,
            op_payload={"kind": "delete", "pos": 5},
        )

        self.assertEqual(result.status, "ok")
        self.assertEqual(result.ack["server_version"], 5)
        self.assertEqual(result.ack["op"], {"kind": "delete", "pos": 9})
        self.assertEqual(doc.content(), "hello!!!!world")

    def test_same_character_delete_becomes_noop(self):
        doc = CollaborativeDocument("abc")
        first = doc.process_operation(
            client_id="A",
            op_id="A-1",
            base_version=0,
            op_payload={"kind": "delete", "pos": 1},
        )
        second = doc.process_operation(
            client_id="B",
            op_id="B-1",
            base_version=0,
            op_payload={"kind": "delete", "pos": 1},
        )
```

```python

        self.assertEqual(first.status, "ok")
        self.assertEqual(second.status, "ok")
        self.assertEqual(second.ack["op"]["kind"], "noop")
        self.assertEqual(doc.content(), "ac")
        self.assertEqual(doc.version, 2)

    def test_history_expiry_requests_resync(self):
        doc = CollaborativeDocument("", history_limit=2)
        for version in range(3):
            result = doc.process_operation(
                client_id="A",
                op_id=f"A-{version}",
                base_version=version,
                op_payload={"kind": "insert", "pos": version, "char": "x"},
            )
            self.assertEqual(result.status, "ok")

        self.assertEqual(doc.history_start_version, 1)
        expired = doc.process_operation(
            client_id="B",
            op_id="B-1",
            base_version=0,
            op_payload={"kind": "insert", "pos": 0, "char": "y"},
        )

        self.assertEqual(expired.status, "resync")
        self.assertEqual(expired.messages[0]["type"], "RESYNC_REQUIRED")
        self.assertEqual(expired.messages[1]["type"], "DOC_STATE")

    def test_server_version_is_strictly_incremented(self):
        doc = CollaborativeDocument("")
        versions = []
        for index, char in enumerate("abc"):
            result = doc.process_operation(
                client_id="A",
                op_id=f"A-{index}",
                base_version=index,
                op_payload={"kind": "insert", "pos": index, "char": char},
            )
            versions.append(result.ack["server_version"])

        self.assertEqual(versions, [1, 2, 3])
        self.assertEqual(doc.content(), "abc")

    def test_insert_chinese_character(self):
        doc = CollaborativeDocument("")
```

```
        result = doc.process_operation(
            client_id="A",
            op_id="A-1",
            base_version=0,
            op_payload={"kind": "insert", "pos": 0, "char": " 你"},
        )

        self.assertEqual(result.status, "ok")
        self.assertEqual(result.ack["server_version"], 1)
        self.assertEqual(doc.content(), " 你")
```

8.11 客户端队列测试

客户端状态测试验证了 ACK 驱动发送队列、远程版本缺口处理、服务端分配客户端编号、中文字符输入和宽字符删除等行为。

```
class ClientSyncStateTests(unittest.TestCase):
    def test_pending_queue_sends_next_only_after_ack(self):
        sent = []
        state = ClientSyncState("u1", sent.append)
        state.set_document("abc", 0)

        first_id = state.queue_operation(Operation("insert", 3, "x"))
        second_id = state.queue_operation(Operation("insert", 4, "y"))

        self.assertEqual(len(sent), 1)
        self.assertEqual(sent[0]["op_id"], first_id)
        self.assertEqual(sent[0]["base_version"], 0)
        self.assertEqual(state.inflight_op.op_id, first_id)
        self.assertEqual(len(state.pending_queue), 1)

        state.handle_message(
            {
                "type": "ACK",
                "op_id": first_id,
                "version": 1,
                "op": {"kind": "insert", "pos": 3, "char": "x"},
            }
        )

        self.assertEqual(state.content, "abcx")
        self.assertEqual(len(sent), 2)
        self.assertEqual(sent[1]["op_id"], second_id)
        self.assertEqual(sent[1]["base_version"], 1)

```

```python
        state.handle_message(
            {
                "type": "ACK",
                "op_id": second_id,
                "version": 2,
                "op": {"kind": "insert", "pos": 4, "char": "y"},
            }
        )

        self.assertEqual(state.content, "abcxy")
        self.assertIsNone(state.inflight_op)
        self.assertEqual(len(state.pending_queue), 0)

    def test_remote_gap_requests_document(self):
        sent = []
        state = ClientSyncState("u1", sent.append)
        state.set_document("", 0)
        state.handle_message(
            {
                "type": "REMOTE_OP",
                "version": 3,
                "op": {"kind": "insert", "pos": 0, "char": "z"},
            }
        )

        self.assertEqual(sent[-1], {"type": "REQUEST_DOC"})

    def test_welcome_updates_server_assigned_client_id(self):
        sent = []
        state = ClientSyncState("u1", sent.append)
        state.handle_message({"type": "WELCOME", "client_id": "u2"})
        state.set_document("", 0)
        op_id = state.queue_operation(Operation("insert", 0, "x"))

        self.assertEqual(state.client_id, "u2")
        self.assertEqual(op_id, "u2-1")
        self.assertEqual(sent[0]["client_id"], "u2")

    def test_chinese_character_is_insertable(self):
        self.assertTrue(is_insertable_character(" 你"))
        self.assertFalse(is_insertable_character("\x13"))
        self.assertFalse(is_insertable_character("\t"))

    def test_chinese_display_width_helpers(self):
        self.assertEqual(text_display_width("a 你 b"), 4)
        self.assertEqual(display_col_to_char_col("a 你 b", 2), 1)
        self.assertEqual(display_col_to_char_col("a 你 b", 3), 2)
```

```python

    def test_wide_character_backspace_queues_delete(self):
        sent = []
        client = TerminalClient("127.0.0.1", 8765, "u1")
        client.state = ClientSyncState("u1", sent.append)
        client.state.set_document("ab", 0)
        client.cursor_pos = 2

        client._handle_key("\x7f")

        self.assertIn("\x7f", BACKSPACE_KEYS)
        self.assertEqual(sent[0]["op"], {"kind": "delete", "pos": 1})
        self.assertEqual(client.cursor_pos, 1)
```

Linux 平台下的基本命令

作业中要求使用 Linux 平台进行操作，我使用一台阿里云 ECS 服务器完成实验。服务器配置为 2 核 2 GB，操作系统为 Ubuntu 24.04。本次实验通过 SSH 连接到服务器，并在真实 Linux 环境中完成命令演示。我使用 Termius 作为 SSH 客户端，便于在 Windows 电脑上远程连接服务器。

本节按照文件和目录、文件内容处理、系统信息、权限管理以及管道组合五类来整理命令。这样比单纯罗列命令更容易看出不同命令的使用场景，也能体现 Linux 命令之间可以组合使用的特点。

1 文件和目录操作

1. 查看当前所在目录：`pwd`
2. 查看当前目录下的文件：`ls`
3. 以详细形式查看文件：`ls -l`
4. 查看隐藏文件：`ls -a`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# pwd
/root
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  desktop/  n2n/       snap/
clashctl/  derp/                     frp/      rustDesk/  task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls -l
total 40
drwxr-xr-x  2 root root 4096 Feb  9 14:25 alist/
drwxr-xr-x  7 root root 4096 Mar  2 17:35 clashctl/
drwxr-xr-x  6 root root 4096 Mar  2 17:35 clash-for-linux-install/
drwxr-xr-x  2 root root 4096 Mar  9 19:47 derp/
drwxr-xr-x  2 root root 4096 Apr 27 11:27 desktop/
drwxr-xr-x  3 root root 4096 Feb 21 12:31 frp/
drwxr-xr-x 17 root root 4096 Feb  9 22:49 n2n/
drwxr-xr-x  3 root root 4096 Mar  2 21:18 rustDesk/
drwx------  3 root root 4096 Mar 14 12:23 snap/
drwxr-xr-x  2 root root 4096 Apr 25 20:50 task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls -a
./             .cache/                  desktop/  .pip/             .ssh/
../            clashctl/                frp/      .profile          task2/
alist/         clash-for-linux-install/ .lesshst  .pydistutils.cfg  .vim/
.bash_history  .config/                 .local/   rustDesk/         .viminfo
.bashrc        derp/                    n2n/      snap/             .wget-hsts
```

图 14 命令 1.1-1.4

5. 创建一个目录：`mkdir linux_test`
6. 进入目录：`cd linux_test`
7. 创建空文件：`touch file1.txt`
8. 向文件写入内容：`echo "hello linux" > file1.txt`

9. 查看文件内容：`cat file1.txt`
10. 复制文件：`cp file1.txt file2.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir linux_test
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd linux_test/
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# touch file1.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# echo "hello linux" > file1.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cat file1.txt
hello linux
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cp file1.txt file2.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cat file2.txt
hello linux
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# ls
file1.txt  file2.txt
```

图 15 命令 1.5-1.10

11. 重命名文件：`mv file1.txt newfile.txt`
12. 删除文件：`rm newfile.txt`
13. 返回上一级目录：`cd ..`
14. 删除目录：`rm -r linux_test`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# mv file1.txt  newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# ls
file2.txt  newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# rm newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# ls
file2.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_test# cd ..
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  desktop/  linux_test/  rustDesk/  task2/
clashctl/  derp/                     frp/      n2n/         snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# rm -r linux_test/
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  desktop/  n2n/       snap/
clashctl/  derp/                     frp/      rustDesk/  task2/
```

图 16 命令 1.11-1.14

2 文件内容查看与处理命令

我先使用如下命令创建测试目录和测试文件，然后再对文件内容处理命令进行演示。

```
mkdir linux_homework
cd linux_homework
echo -e "apple\nbanana\napple\norange\nbanana\nlinux\ncommand\npipe" > words.txt
```

1. 查看完整内容：`cat words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir linux_homework
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  desktop/  linux_homework/  rustDesk/  task2/
clashctl/  derp/                     frp/      n2n/             snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd linux_homework/
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# echo -e "apple\nbanana\napple\norange\nbanana\n
linux\ncommand\npipe" > words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls
words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt
apple
banana
apple
orange
banana
linux
command
pipe
```

图 17 命令 2.1

2. 查看文件前 3 行：`head -n 3 words.txt`
3. 查看文件后 3 行：`tail -n 3 words.txt`
4. 统计文件行数、单词数、字符数：`wc words.txt`
5. 统计文件行数：`wc -l words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# head -n 3 words.txt
apple
banana
apple
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# tail -n 3 words.txt
linux
command
pipe
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# wc words.txt
 8  8 52 words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# wc -l words.txt
8 words.txt
```

图 18 命令 2.2-2.5

6. 对文件内容排序：`sort words.txt`
7. 去除相邻重复行：`uniq words.txt`
8. 查找包含 apple 的行：`grep "apple" words.txt`
9. 查找包含 banana 的行，并显示行号：`grep -n "banana" words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# sort words.txt
apple
apple
banana
banana
command
linux
orange
pipe
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# uniq words.txt
apple
banana
apple
orange
banana
linux
command
pipe
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# grep "banana" words.txt
banana
banana
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# grep -n "banana" words.txt
2:banana
5:banana
```

图 19 命令 2.6-2.9

`uniq` 只能去除相邻的重复行，如果重复内容之间隔着其他行，直接执行 `uniq words.txt` 并不能完成全局去重。因此在后面的管道命令中，我又使用 `sort words.txt | uniq`，先把相同内容排到相邻位置，再进行去重统计。

3 系统信息查看命令

1. 查看系统内核和系统信息：`uname -a`
2. 查看当前用户名：`whoami`
3. 查看当前日期时间：`date`
4. 显示日历：`cal`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# uname -a
Linux iZbp11a51jwdv4pjbv2r1oZ 6.8.0-90-generic #91-Ubuntu SMP PREEMPT_DYNAMIC Tue Nov 18 14:14
:30 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# whoami
root
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# date
Mon Apr 27 11:36:17 AM CST 2026
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cal
     April 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30
```

图 20 命令 3.1-3.4

如果系统默认没有安装 `cal` 命令，需要先安装 `ncal` 包。我使用 `sudo apt install`

`ncal` 安装之后，就可以正常显示日历。

5. 查看磁盘空间：`df -h`

6. 查看内存使用情况：`free -h`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           162M  1.5M  160M   1% /run
efivarfs        256K  7.4K  244K   3% /sys/firmware/efi/efivars
/dev/vda3        40G  5.8G   32G  16% /
tmpfs           807M     0  807M   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/vda2       197M  6.2M  191M   4% /boot/efi
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/f6fdeb7a0935974bbc5df63
262aeeeb1eab665b7deb467c5473618aaa7edeb16
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/237c5d820903ac7889c5633
f264a81a5cfbd5beb6d68c2e52780dcf004e0fbc2
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/8b23e10ea750512823dc7b9
8534178b2485c5248054f51408fb1b2651dcba3a4
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/27db7f8296651a194aea4a0
77a1dfe1e2ca5ca9a74782f6885fae7792df73f73
tmpfs           162M   12K  162M   1% /run/user/0
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# free -h
               total        used        free      shared  buff/cache   available
Mem:           1.6Gi       651Mi       137Mi       3.3Mi       994Mi       961Mi
Swap:             0B          0B          0B
```

图 21　命令 3.5-3.6

7. 查看系统进程和资源占用：`top`

```
Myserver
top - 16:00:00 up 75 days, 23:44,  2 users,  load average: 0.01, 0.02, 0.00
Tasks: 141 total,   1 running, 140 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  4.8 sy,  0.0 ni, 95.2 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1613.0 total,    207.5 free,    688.7 used,    884.0 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    924.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   22940  13056   8576 S   0.0   0.8  15:38.32 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:01.70 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:27.08 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   3:25.52 rcu_preempt
     18 root      rt   0       0      0      0 S   0.0   0.0   0:18.02 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     22 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
     23 root      rt   0       0      0      0 S   0.0   0.0   0:16.23 migration/1
     24 root      20   0       0      0      0 S   0.0   0.0   0:21.53 ksoftirqd/1
     26 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
```

图 22　命令 3.7

8. 查看当前系统进程：`ps aux`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.7  22940 13056 ?        Ss   Feb08  15:38 /usr/lib/systemd/systemd --system --
root           2  0.0  0.0      0     0 ?        S    Feb08   0:01 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Feb08   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-rcu_p]
root           6  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-slub_]
root           7  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-netns]
root           9  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/0:0H-events_highpri]
root          12  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-mm_pe]
root          13  0.0  0.0      0     0 ?        I    Feb08   0:00 [rcu_tasks_kthread]
root          14  0.0  0.0      0     0 ?        I    Feb08   0:00 [rcu_tasks_rude_kthread]
root          15  0.0  0.0      0     0 ?        I    Feb08   0:00 [rcu_tasks_trace_kthread]
root          16  0.0  0.0      0     0 ?        S    Feb08   0:27 [ksoftirqd/0]
root          17  0.0  0.0      0     0 ?        I    Feb08   3:25 [rcu_preempt]
root          18  0.0  0.0      0     0 ?        S    Feb08   0:18 [migration/0]
root          19  0.0  0.0      0     0 ?        S    Feb08   0:00 [idle_inject/0]
root          20  0.0  0.0      0     0 ?        S    Feb08   0:00 [cpuhp/0]
root          21  0.0  0.0      0     0 ?        S    Feb08   0:00 [cpuhp/1]
root          22  0.0  0.0      0     0 ?        S    Feb08   0:00 [idle_inject/1]
root          23  0.0  0.0      0     0 ?        S    Feb08   0:16 [migration/1]
root          24  0.0  0.0      0     0 ?        S    Feb08   0:21 [ksoftirqd/1]
root          26  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/1:0H-events_highpri]
root          27  0.0  0.0      0     0 ?        S    Feb08   0:00 [kdevtmpfs]
root          28  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-inet_]
root          29  0.0  0.0      0     0 ?        S    Feb08   0:02 [kauditd]
root          31  0.0  0.0      0     0 ?        S    Feb08   0:04 [khungtaskd]
root          33  0.0  0.0      0     0 ?        S    Feb08   0:00 [oom_reaper]
```

图 23 命令 3.8

由于使用的是远程服务器，系统进程比较多，截图中只保留了与本次演示相关的部分。

9. 查看历史命令：`history`

```
 801  uname -a
 802  whoami
 803  date
 804  cal
 805  apt install ncal
 806  cal
 807  clear
 808  uname -a
 809  whoami
 810  date
 811  cal
 812  clea
 813  clear
 814  df -h
 815  top
 816  df -h
 817  clear
 818  df -h
 819  clear
 820  top
 821  clear
 822  free -h
 823  df -h
 824  free -h
 825  clear
 826  ps aux
 827  clear
 828  history
```

图 24 命令 3.9

历史记录中包含了我之前使用过的命令，包括一些测试命令和安装命令等。这里只截了本次课程展示的几个历史命令。

4 权限相关命令

1. 创建脚本文件：`touch test.sh`
2. 查看文件权限：`ls -l test.sh`
3. 给文件添加可执行权限：`chmod +x test.sh`
4. 再次查看权限变化：`ls -l test.sh`
5. 查看文件详细状态信息：`stat test.sh`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# touch test.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls -l test.sh
-rw-r--r-- 1 root root 0 Apr 27 11:38 test.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# chmod +x test.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls -l test.sh
-rwxr-xr-x 1 root root 0 Apr 27 11:38 test.sh*
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# stat test.sh
  File: test.sh
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: 253,3   Inode: 1179141     Links: 1
Access: (0755/-rwxr-xr-x)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-04-27 11:38:00.613626249 +0800
Modify: 2026-04-27 11:38:00.613626249 +0800
Change: 2026-04-27 11:38:14.493042638 +0800
 Birth: 2026-04-27 11:38:00.613626249 +0800
```

图 25 命令 4.1-4.5

5 管道符组合命令

管道符 | 的作用是将前一个命令的输出结果作为后一个命令的输入。通过管道符，可以把多个简单命令组合成一个更复杂的数据处理流程。例如 `cat words.txt | sort | uniq -c | sort -nr` 先读取文件内容，再进行排序，然后统计重复行出现次数，最后按照出现次数从高到低排序，体现了 Linux 命令组合使用的灵活性。

1. 把文件内容传给 sort 排序后去重：`cat words.txt | sort | uniq`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | sort | uniq
apple
banana
command
linux
orange
pipe
```

图 26 命令 5.1

2. 统计每个单词出现次数：`cat words.txt | sort | uniq -c`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | sort | uniq -c
      2 apple
      2 banana
      1 command
      1 linux
      1 orange
      1 pipe
```

图 27　命令 5.2

3. 找出包含字母 a 的行，并排序：`grep "a" words.txt | sort`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | grep "a" | sort
apple
apple
banana
banana
command
orange
```

图 28　命令 5.3

4. 在当前目录中筛选.txt 文件：`ls | grep '\.txt$'`
5. 在进程列表中查找 bash 相关进程：`ps aux | grep bash`
6. 查看最近使用的命令：`history | tail`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ls -l | grep ".txt"
-rw-r--r-- 1 root root 52 Apr 27 11:33 words.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# ps aux | grep bash
root      219565  0.0  0.0   2616   128 ?        Ss   Mar09   0:00 /bin/sh -c bash /app/build_
cert.sh $DERP_HOST $DERP_CERTS /app/san.conf &&     /app/derper --hostname=$DERP_HOST     --ce
rtmode=manual     --certdir=$DERP_CERTS     --stun=$DERP_STUN      --a=$DERP_ADDR     --http-p
ort=$DERP_HTTP_PORT     --verify-clients=$DERP_VERIFY_CLIENTS
root      243312  0.0  0.2   9052  4480 tty1     S+   Mar14   0:00 -bash
root      578530  0.0  0.1   6544  2304 pts/0    S+   11:40   0:00 grep --color=auto bash
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# history | tail
/etc/ssh/sshd_config | grep -E '^Port'
sudo systemctl status ssh
service ssh restart
ufw status
systemctl restart ssh
systemctl status ssh
sudo systemctl list-units --type=service | grep ssh
systemctl restart sshd
cd root
ranger
```

图 29　命令 5.4-5.6

7. 筛选磁盘挂载信息：`df -h | grep "/dev"`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# df -h | grep "/"
tmpfs           162M  1.5M  160M   1% /run
efivarfs        256K  7.4K  244K   3% /sys/firmware/efi/efivars
/dev/vda3        40G  5.8G   32G  16% /
tmpfs           807M     0  807M   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/vda2       197M  6.2M  191M   4% /boot/efi
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/f6fdeb7a0935974bbc5df63
262aeeeb1eab665b7deb467c5473618aaa7edeb16
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/237c5d820903ac7889c5633
f264a81a5cfbd5beb6d68c2e52780dcf004e0fbc2
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/8b23e10ea750512823dc7b9
8534178b2485c5248054f51408fb1b2651dcba3a4
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/27db7f8296651a194aea4a0
77a1dfe1e2ca5ca9a74782f6885fae7792df73f73
tmpfs           162M   12K  162M   1% /run/user/0
```

图 30 命令 5.7

8. 统计文件行数：`cat words.txt | wc -l`
9. 统计单词出现次数，并按出现次数从高到低排序：`cat words.txt | sort | uniq -c | sort -nr`

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | wc -l
8
root@iZbp11a51jwdv4pjbv2r1oZ ~/linux_homework# cat words.txt | sort | uniq -c | sort -nr
      2 banana
      2 apple
      1 pipe
      1 orange
      1 linux
      1 command
```

图 31 命令 5.8-5.9

6 实验小结

通过本次基础命令实验，我对 Linux 命令行的工作方式有了更系统的理解。文件和目录命令主要解决“在哪里”和“有什么”的问题，内容查看与处理命令用于观察和整理文本，系统信息命令帮助了解当前运行环境，权限命令则体现了 Linux 中文件可读、可写、可执行权限的基本模型。

其中管道命令给我的印象最深。单个命令的功能通常比较简单，但通过 `|` 可以把多个命令连接成完整的数据处理流程。例如排序、去重、计数和再次排序可以组合成一条命令完成词频统计。这体现了 Linux 工具“每个程序只做好一件事，再通过组合完成复杂任务”的思想，也为后续 Shell 脚本编程打下了基础。

Linux 下的 Shell 编程案例

本次作业我设计并实现了一个基于 Bash Shell 的 Linux 命令答题系统。程序通过菜单提供开始答题、查看历史成绩、清空历史成绩和退出系统等功能，题目从外部题库文件中读取，答题结束后会把本次成绩保存到成绩文件中。这个案例既能练习 Linux 基础命令，也能体现 Shell 脚本中的条件判断、循环控制、函数封装、文件读写和程序调试等知识点。相比只演示单条命令，答题系统需要把输入、判断、统计、持久化和异常处理组织成完整流程，更能体现 Shell 脚本在 Linux 自动化任务中的作用。

1 程序功能说明

本程序的主要功能如下：

- 显示主菜单，用户可以通过输入编号选择不同功能。
- 从 `questions.txt` 题库文件中逐行读取题目和标准答案。
- 判断用户输入是否为空、是否与标准答案一致，并根据结果计算得分和错误次数。
- 当答错次数达到设定上限时，系统自动结束本轮答题。
- 将每次答题的时间、得分、已答题数和错误题数保存到 `score.txt` 文件。
- 支持查看历史成绩，并使用 `awk` 统计历史最高分。
- 支持清空历史成绩，清空前需要用户再次确认，避免误操作。

程序采用“主菜单 + 功能函数”的结构，整体逻辑比较清晰。用户进入程序后，主循环会不断显示菜单，直到用户选择退出。每个功能被封装成独立函数，例如 `init_env`、`show_menu`、`start_quiz`、`save_score`、`show_history` 和 `clear_history`，这样可以减少重复代码。

2 Shell 脚本执行方法

在 Linux 平台下，可以进入脚本所在目录后执行以下命令运行程序：

```
cd task2
chmod +x linux_quiz.sh
./linux_quiz.sh
```

其中，`chmod +x linux_quiz.sh` 用于给脚本添加可执行权限，`./linux_quiz.sh` 表示执行当前目录下的脚本文件。

3 程序实现思路

脚本整体采用函数化结构，把初始化环境、显示菜单、开始答题、保存成绩、查看历史成绩和清空历史成绩拆分为多个函数。主程序只负责循环显示菜单并根据用户选择调用对应函数，这样可以避免所有逻辑堆在一个长脚本中，也方便后续单独修改某个功能。

题库文件使用 `|` 作为题目和答案之间的分隔符，每一行表示一道题。答题函数通过 `while IFS='|' read -r question correct_answer` 逐行读取题库，并使用独立文件描述符读取文件，避免用户输入和题库读取互相干扰。用户输入后，脚本会先去掉首尾空格，再判断是否为空、是否与标准答案一致，并更新分数、已答题数和错误次数。当错误次数达到上限时，当前轮答题自动结束。

成绩文件使用普通文本保存，每条记录包含答题时间、得分、已答题数和错误题数。查看历史成绩时，脚本使用 `awk` 从成绩文件中统计最高分；清空历史成绩前会要求用户输入确认，避免误删。这样既保持了脚本实现的简单性，也展示了 Shell 脚本对文本文件和命令行工具的组合能力。

4 案例展示

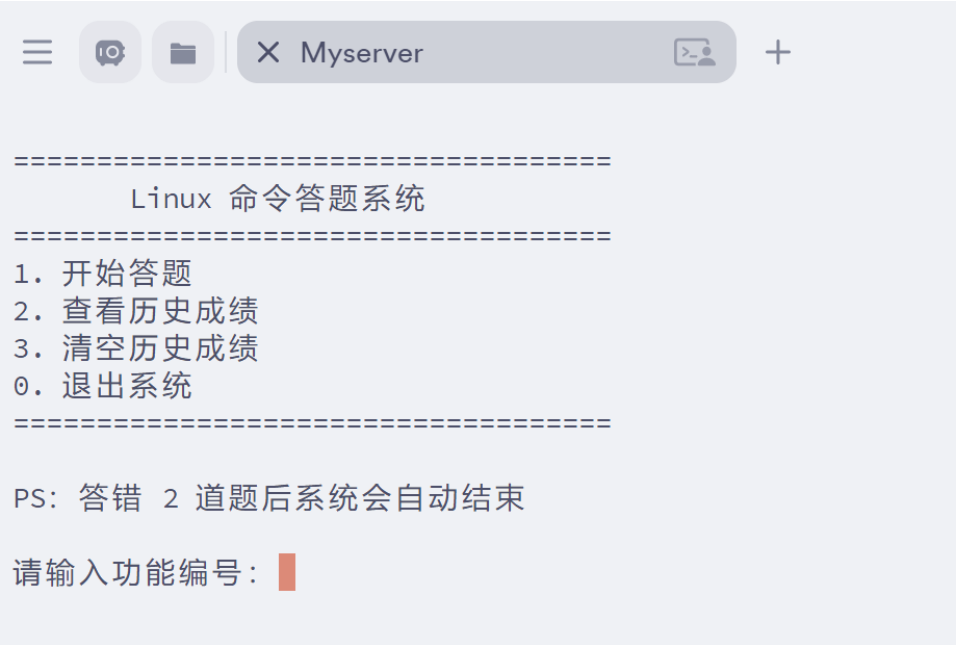
与上一次基础命令作业一致，本次 Shell 脚本也在 Ubuntu 服务器上运行和测试，后续调试同样在这台服务器上完成。

首先创建一个目录来存放本次作业，然后放入 Shell 脚本文件和 `txt` 题库文件。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/  clashctl/  clash-for-linux-install/  derp/  frp/  linux_homework/  n2n/  rustDesk/  snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir task2
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  frp/             n2n/       snap/
clashctl/  derp/                     linux_homework/  rustDesk/  task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh  questions.txt
```

随后按照前面的执行方法运行脚本。

主菜单页面：



开始答题界面：

图 32 答题界面

答题结束界面：

```
-----------------------------------
第 11 题：查看当前登录用户
请输入答案：uname
回答错误！正确答案是：whoami
当前得分：9，错误次数：2/2


错误次数达到 2 次，答题结束。

========== 答题结果 ==========
本次得分：9
已答题数：11
错误题数：2
成绩已保存到 ./score.txt

按 Enter 键返回主菜单...
```

图 33 答题结束界面

历史成绩界面：

```
========== 历史成绩 ==========
最高分：24
-------------------------------------
时间 | 得分 | 已答题数 | 错误题数
-------------------------------------
2026-04-25 21:07:00 | 1 | 1 | 2
2026-04-27 13:17:50 | 9 | 11 | 2
2026-04-27 13:20:38 | 1 | 1 | 2
2026-04-27 13:29:59 | 24 | 25 | 1

按 Enter 键返回主菜单...
```

图 34 历史成绩

题库和历史成绩都保存在当前目录下的 `txt` 文件中。目前题库包含 25 道题，后续如果需要扩展题库，可以直接在 `questions.txt` 文件中添加题目和答案，格式为“题目 | 答案”，每行一题。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  questions.txt*  score.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# cat score.txt
2026-04-25 21:07:00|1|1|2
2026-04-27 13:17:50|9|11|2
2026-04-27 13:20:38|1|1|2
2026-04-27 13:29:59|24|25|1
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# cat questions.txt
查看当前所在目录|pwd
查看当前目录下文件和文件夹|ls
切换目录|cd
创建目录|mkdir
创建空文件|touch
查看文件内容|cat
```

图 35 题库和历史成绩文本文件

清空历史成绩时，程序会要求用户确认，可以选择确认清空或取消操作。

```
====================================
       Linux 命令答题系统
====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
====================================

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 3

确认清空历史成绩吗? 输入 y 确认: n
已取消清空操作。

按 Enter 键返回主菜单...
```

```
====================================
       Linux 命令答题系统
====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
====================================

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 3

确认清空历史成绩吗? 输入 y 确认: y
历史成绩已清空。

按 Enter 键返回主菜单...
```

图 36 清空历史成绩

退出系统。

```
=====================================
       Linux 命令答题系统
=====================================
1. 开始答题
2. 查看历史成绩
3. 清空历史成绩
0. 退出系统
=====================================

PS: 答错 2 道题后系统会自动结束

请输入功能编号: 0
系统已退出。
```

图 37 退出系统

5 Shell 程序调试方法

在开发和调试 Shell 脚本时，我主要使用 `bash -n linux_quiz.sh` 命令来检查脚本的语法错误。这个命令会解析脚本但不执行它，可以快速发现语法问题。

正常情况下，命令行没有输出，表示没有发现语法错误。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# bash -n linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2#
```

如果有错误，会显示错误信息和所在行号，方便定位问题。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# bash -n linux_quiz.sh
linux_quiz.sh: line 143: syntax error near unexpected token `}'
linux_quiz.sh: line 143: `}'
```

使用 `vim` 打开脚本后可以看到，错误原因是分支结构中少了一个 `fi`，导致语法检查失败。补全该关键字后再次运行检查，脚本就不再报错。

```
# 查看历史成绩，并在最上方显示最高分
show_history() {
    echo
    echo "========== 历史成绩 ==========="

    if [ ! -f "$SCORE_FILE" ]; then
        echo "暂无成绩文件。"
        return
    fi

    if [ ! -s "$SCORE_FILE" ]; then
        echo "暂无历史成绩。"
        return
```

图 38 错误修正

完成 Shell 脚本之后，可以使用 `bash -x linux_quiz.sh` 来执行脚本并显示每条命令的执行过程，这样可以更细致地调试程序逻辑和变量值。

```
Myserver
+ show_menu
+ echo

+ echo ====================================
====================================
+ echo '       Linux 命令答题系统'
       Linux 命令答题系统
+ echo ====================================
====================================
+ echo '1. 开始答题'
1. 开始答题
+ echo '2. 查看历史成绩'
2. 查看历史成绩
+ echo '3. 清空历史成绩'
3. 清空历史成绩
+ echo '0. 退出系统'
0. 退出系统
+ echo ====================================
====================================
+ echo

+ echo 'PS: 答错 2 道题后系统会自动结束'
PS: 答错 2 道题后系统会自动结束
+ echo

+ read -r -p 请输入功能编号:  choice
请输入功能编号:
```

图 39 执行跟踪调试

另外，我还构造了一些异常输入场景进行测试，例如删除 `questions.txt`、输入非法选项等，用来验证程序的健壮性和错误处理能力。

删除 `questions.txt` 后运行程序：

```
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  questions.txt*  score.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# rm questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh*  score.txt
```

图 40 删除题库文件

```
题库文件不存在: ./questions.txt

按 Enter 键返回主菜单...
```

图 41 运行程序

可以看出程序并没有崩溃，而是提示了题库文件不存在，说明程序的错误处理机制是有效的。

恢复题库文件后，答题输入中文选项：

```
-----------------------------------
第 1 题: 查看当前所在目录
请输入答案: 中文
回答错误! 正确答案是: pwd
当前得分: 0, 错误次数: 1/2

按 Enter 继续下一题...
```

图 42 输入中文选项

程序能够正确识别并说明该答案错误，说明程序对输入的处理也是正确的。

在主菜单页面输入非法选项时，程序不会崩溃，而是继续循环并提示用户输入正确选项。

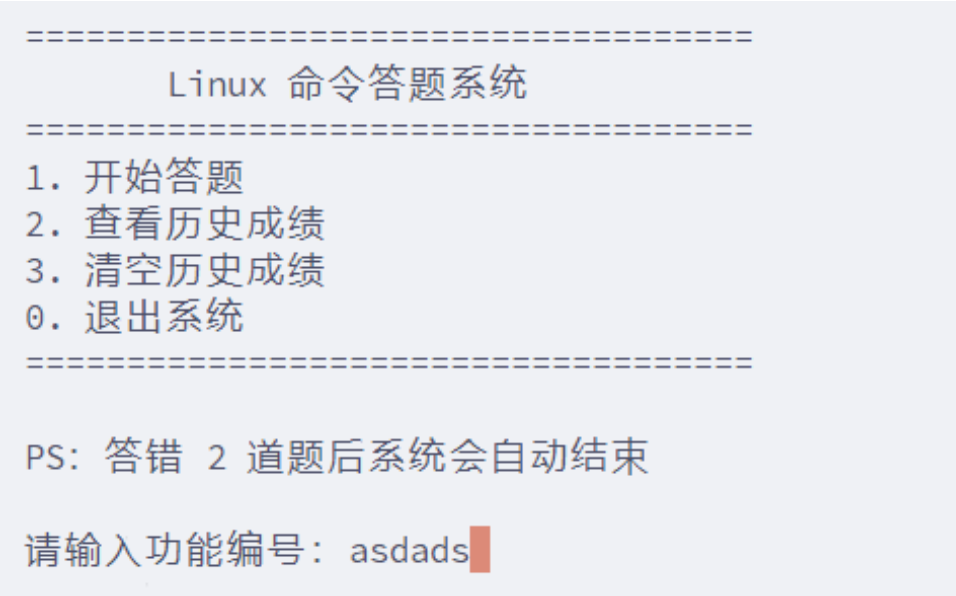


图 43 输入非法选项

6 实验小结

这个 Shell 编程案例把前一节学到的基础命令进一步组织成了一个可交互的小程序。脚本中既有 `read` 获取用户输入，也有 `if` 和 `case` 完成分支判断，还有 `while` 循环读取题库文件，并通过重定向把成绩追加到文本文件中。相比单独练习命令，完整脚本更强调流程设计和异常处理。

通过调试过程可以看到，Shell 脚本虽然语法相对简洁，但对符号和结构非常敏感，例如缺少 `fi` 就会导致语法检查失败。因此在编写脚本时，先用 `bash -n` 检查语法，再用 `bash -x` 跟踪执行过程，是比较实用的调试方法。后续如果继续完善这个答题系统，还可以增加随机出题、错题记录、分数排行榜和题库合法性检查等功能。

7 源代码展示

以下是本次 Shell 编程作业的完整源代码：

```
#!/usr/bin/env bash

QUESTION_FILE="./questions.txt"
SCORE_FILE="./score.txt"
WRONG_LIMIT=2

# 初始化运行环境
init_env() {
    if [ ! -f "$SCORE_FILE" ]; then
```

```
        touch "$SCORE_FILE"
    fi
}

# 暂停，便于用户查看结果
pause_screen() {
    echo
    read -r -p " 按 Enter 键返回主菜单..."
}

# 显示主菜单
show_menu() {
    echo
    echo "===================================="
    echo "        Linux 命令答题系统"
    echo "===================================="
    echo "1. 开始答题"
    echo "2. 查看历史成绩"
    echo "3. 清空历史成绩"
    echo "0. 退出系统"
    echo "===================================="
    echo
    echo "PS: 答错 $WRONG_LIMIT 道题后系统会自动结束"
    echo
}

# 核心答题功能
start_quiz() {
    score=0
    wrong=0
    total=0
    answered=0

    if [ ! -f "$QUESTION_FILE" ]; then
        echo " 题库文件不存在: $QUESTION_FILE"
        return
    fi

    # 使用文件描述符 3 读取题库，避免用户输入被题库文件提前消费。
    while IFS='|' read -r question correct_answer <&3; do
        # 跳过空行
        if [ -z "$question" ]; then
            continue
        fi

        total=$((total + 1))

```

```
        echo "-----------------------------------"
        echo " 第 $total 题: $question"
        read -r -p " 请输入答案: " user_answer

        # 去掉用户输入前后的空格
        user_answer="$(echo "$user_answer" | xargs)"

        if [ -z "$user_answer" ]; then
            echo " 答案不能为空，本题判定为错误。"
            wrong=$((wrong + 1))
        elif [ "$user_answer" = "$correct_answer" ]; then
            echo " 回答正确，+1 分。"
            score=$((score + 1))
            answered=$((answered + 1))
        else
            echo " 回答错误！正确答案是: $correct_answer"
            wrong=$((wrong + 1))
            answered=$((answered + 1))
        fi

        echo " 当前得分: $score，错误次数: $wrong/$WRONG_LIMIT"
        echo

        if [ "$wrong" -ge "$WRONG_LIMIT" ]; then
            echo
            echo " 错误次数达到 $WRONG_LIMIT 次，答题结束。"
            break
        fi

        read -r -p " 按 Enter 继续下一题..."
        clear

    done 3< "$QUESTION_FILE"

    echo
    echo "========== 答题结果 ==========="
    echo " 本次得分: $score"
    echo " 已答题数: $answered"
    echo " 错误题数: $wrong"

    save_score "$score" "$answered" "$wrong"
}

# 保存成绩
save_score() {
    current_score="$1"
    answered_count="$2"
```

```
    wrong_count="$3"
    current_time="$(date '+%Y-%m-%d %H:%M:%S')"

    echo "$current_time|$current_score|$answered_count|$wrong_count" >> "$SCORE_FILE"

    if [ -w "$SCORE_FILE" ]; then
        echo " 成绩已保存到 $SCORE_FILE"
    else
        echo " 成绩文件不可写，保存失败。"
    fi
}

# 查看历史成绩，并在最上方显示最高分
show_history() {
    echo
    echo "========== 历史成绩 ==========="

    if [ ! -f "$SCORE_FILE" ]; then
        echo " 暂无成绩文件。"
        return
    fi

    if [ ! -s "$SCORE_FILE" ]; then
        echo " 暂无历史成绩。"
        return
    fi

    highest_score="$(awk -F'|' 'BEGIN {max=0} {if ($2 > max) max=$2} END {print
    ↪ max}' "$SCORE_FILE")"

    echo " 最高分: $highest_score"
    echo "-----------------------------------"
    echo " 时间 | 得分 | 已答题数 | 错误题数"
    echo "-----------------------------------"

    while IFS='|' read -r time score answered wrong; do
        if [ -n "$time" ]; then
            echo "$time | $score | $answered | $wrong"
        fi
    done < "$SCORE_FILE"
}

# 清空历史成绩
clear_history() {
    echo
    read -r -p " 确认清空历史成绩吗？输入 y 确认: " confirm

```

```
    if [ "$confirm" = "y" ] || [ "$confirm" = "Y" ]; then
        > "$SCORE_FILE"
        echo " 历史成绩已清空。"
    else
        echo " 已取消清空操作。"
    fi
}

# 主程序入口
init_env

while true; do
    clear
    show_menu
    read -r -p " 请输入功能编号: " choice

    case "$choice" in
        1)
            clear
            start_quiz
            pause_screen
            ;;
        2)
            clear
            show_history
            pause_screen
            ;;
        3)
            clear_history
            pause_screen
            ;;
        0)
            echo " 系统已退出。"
            exit 0
            ;;
        *)
            echo " 输入错误，请输入 0 到 3 之间的数字。"
            ;;
    esac
done
```